h e p i a
Haute école du paysage, d'ingénierie
et d'architecture de Genève

Hes·SO GENÈVE
Haute Ecole Spécialisée
de Suisse occidentale

FPGA Acceleration of DNA Strand Clustering



Bachelor project presented by

Adrian Spycher

Computer science and communication systems with orientation in embedded computer systems

August, 2025

Teacher HES in charge

Laurent Gantel

Client

UNIGE, Pierre-Yves Burgi,
Deputy Director at DiSTIC, Head of the RISe Department

Source of cover illustration :
https://www.the-scientist.com/making-dna-data-storage-a-reality-30218

Table of Contents

Acknowledgments ..... **VI**

Summary ..... **VII**

List of Acronyms ..... **VIII**

List of Figures ..... **X**

List of Tables ..... **XII**

List of Code Snippets ..... **XIII**

List of Appendices ..... **XIV**

Introduction **1**

Chapter 1 : The Edit Distance Algorithm **5**

1.1 The Levenshtein Distance 5

1.2 Myers' Bit-parallel Algorithm 6

Chapter 2 : Algorithms: How to Solve this Problem **12**

2.1 Original Thoughts 12

2.2 OneJoin 12

2.2.a CGK Embedding 13

2.2.b Locality Sensitive Hashing (LSH) 14

2.2.c EmbedJoin 15

2.2.d Parameters 17

2.3 Tag Matcher 19

Chapter 3 : FPGA: Hardware Architecture and Programming **22**

3.1 The Agilex 7 Starter Kit 22

3.2 Open FPGA Stack (OFS) 23

3.2.a Hardware Infrastructure 24

3.2.b Software Infrastructure 25

3.3 Setup 26

Chapter 4 : OneAPI 28

4.1 OneAPI Overview 28

4.2 SYCL and Data Parallel C++ (DPC++) Overview 30

4.3 Advantages of SYCL for FPGA Acceleration 31

4.4 Programming with DPC++ 32

4.4.a Queues and Device Selector 33

4.4.b Parallel For 34

4.4.c Memory Management 34

4.4.d Pragma 37

Chapter 5 : Heterogeneous Programming with CPU and FPGA 41

5.1 Compiling for Mixed Devices 41

5.2 Compiling for FPGA 42

5.3 Compilation Flow 43

Chapter 6 : Implementation Details 46

6.1 Tag Matcher 46

6.1.a Dataflow and Parallelization 46

6.1.b The Myers' Bit-parallel Algorithm 48

6.1.c Levenshtein Distance Considerations 49

6.1.d FPGA Optimization 50

6.2 OneJoin 50

6.2.a oneoligo-a7sk 50

6.2.b Parallelized Part 51

Chapter 7 : Results and Performance 53

7.1 Hardware Used 53

7.2 Tag Matcher 53

7.2.a Precision 53

7.2.b Speed 55

7.2.c FPGA Resource Usage 56
7.3 OneJoin 58
7.3.a Accuracy 58
7.3.b Speed 59
7.3.c FPGA Resource Usage 60
7.4 Proposed Improvements 62
7.4.a Memory Access Optimisations 62
7.4.b Improving the Initiation Interval (II) 63
7.4.c Resource Utilization 63
Chapter 8 : Conclusions and Perspectives 64
Appendix 1 : Relevant links to the project 66
References 67

Acknowledgments

I would like to sincerely thank Laurent Gantel for his continuous guidance and support throughout this project. I am also grateful to Michaël El Kharroubi for carefully reviewing my report and providing valuable advice on difficult tasks. My thanks go as well to Ayenne Sabina Calunsag for revising the English of this work, ensuring its clarity and accuracy. Finally, I deeply appreciate Valentin Bernillon for his constant psychological support, which has been truly encouraging.

Summary

Today, the entire world is accumulating enormous amounts of computer data. This enforces the construction of polluting infrastructure and wastes energy. DNAMIC is a European project that aims to store data in synthetic DNA. To achieve this, a microfactory will be created to read and write on synthetic DNA. This microfactory contains six steps: encoding, synthesis, amplification, storage, sequencing, and decoding. After reading the DNA, errors can appear, either via the reading device or due to DNA mutations. To correct these errors, the PCR technique is used to multiply the DNA. With copies of the DNA, it becomes easier to correct errors. This is where sequence assembly, a sub-step of the microfactory, comes into play, as it's necessary to group the copies relative to their original DNA. Grouping these copies requires calculating their Levenshtein distance. A metric that gives the number of operations required to get from one string a to another string b. These operations are the same as those that can occur in DNA mutations: substitution, addition, and deletion. In order to handle the largest possible amount of data, the highest possible speed is required. The goal of this project is to leverage the speed of FPGAs to accelerate this calculation and by using the OneJoin and Tag Matcher algorithms. Those algorithms are implemented in SYCL, a C++ framework that lets the user program FPGA devices to optimize some function thanks to the High-Level Synthesis (HLS) tools.



Applicant :

Spycher Adrian

Study area : ISC

Teacher in charge :

Laurent Gantel

In collaboration with : UNIGE, DNAMIC

Work subject to a company internship agreement : no

Work subject to a confidentiality agreement : no

List of Acronyms

A7SK	Agilex 7 Starter Kit
AFU	Accelerator Functional Unit
ALUT	Adaptive Look-Up Table
AOT	Ahead-Of-Time
API	Application Programming Interface
ARM	Advanced RISC Machines
ASP	Accelerator Support Package
BMC	Board Management Controller
CGK	Chakraborty-Golden-Koucký
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DDR	Double Data Rate
DFL	Device Feature List
DMA	Direct Memory Access
DNA	Deoxyribonucleic Acid
DPC++	Data Parallel C++
FF	Flip-Flop
FIM	FPGA Interface Manager
FMC+	FPGA Mezzanine Card Plus
FPGA	Field-Programmable Gate Array
GPU	Graphics Processing Unit
HDL	Hardware Description Language
HIP	Heterogeneous-Computing Interface for Portability
HLD	High-Level Design
HLS	High-Level Synthesis

HPS	Hard Processor System
II	Initiation Interval
IOMMU	Input-Output Memory Management Unit
JIT	Just-In-Time
LSH	Locality Sensitive Hashing
M20K	M20K Memory Block
MLAB	Memory Logic Array Block
MMIO	Memory-Mapped Input/Output
OFS	Open FPGA Stack
OPAE	Open Programmable Acceleration Engine
PCIe	PCI Express
PCR	Polymerase Chain Reaction
PR	Partial Reconfiguration
RAM	Random Access Memory
RHEL	Red Hat Enterprise Linux
RTL	Register Transfer Level
SIMT	Single Instruction, Multiple Thread
SPIR-V	Standard Portable Intermediate Representation - Version V
SR-IOV	Single Root I/O Virtualization
SYCL	SYCL (pronounced ‘sick-el’)
SoC	System on Chip
URL	Uniform Resource Locator
USM	Unified Shared Memory
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit
VM	Virtual Machine

List of Figures

Figure 1 Diagram of the six steps constituting the micro-factory of the DNAMIC project [1] 1

Figure 2 The MinION nanopore sequencer by Oxford Nanopore Technology [2] 2

Figure 3 Representation of the Levenshtein Distance on two words [3] 6

Figure 4 Wagner-Fischer matrix between “sitting” and “kitten” [4] 7

Figure 5 Myers’ bit-parallel algorithm dynamic programming (d.p.) matrices [5] 9

Figure 6 Schematic blocks diagram of Myers’ algorithm main loop 11

Figure 7 CGK embedding example 13

Figure 8 LSH example with two DNA strands 15

Figure 9 Tag Matcher algorithm pipeline 21

Figure 10 Block diagram of the Agilex 7 Starter Kits [6] 23

Figure 11 Block diagram of the FPGA Interface Manager (FIM) [7] 24

Figure 12 High level design tool on OFS platforms [7] 29

Figure 13 SYCL logo 30

Figure 14 SYCL implementations and their supported hardware backends [8] 31

Figure 15 A first example of SYCL code explained [9] 32

Figure 16 Schema of a buffer access memory [9] 35

Figure 17 Host CPU and co-processor sharing a single address space for USM allocations [9] 36

Figure 18 Schema of an example code being executed with and without pipeline [10] . 39

Figure 19 Schema of an example code being executed with and without dataflow [10] .40

Figure 20 Basic compilation flow [9] 41

Figure 21 AOT compilation flow [11] 42

Figure 22 Schematic of the compilation flow [12] 44

Figure 23 Tag Matcher algorithm dataflow 46

Figure 24 Confusion matrix of the Tag Matcher algorithm 54

Figure 25 Average time per comparison of the Tag Matcher algorithm across three devices 55

Figure 26 Execution time for parallel bucket generation across three devices 60

List of Tables

Table 1 Different types of edit distance and the operations they allow [13] 5

Table 2 Description of the OneJoin algorithm parameters 17

Table 3 Impacts of parameters on algorithm performance and quality 18

Table 4 Sequencing parameters for `dna-little-processed.txt` 18

Table 5 Versions of components and tools used for the FPGA development 27

Table 6 USM allocation types 36

Table 7 Compilation targets and their characteristics [9] 43

Table 8 Binary encoding of nucleotides for device processing 48

Table 9 Tag Matcher FPGA resource usage 56

Table 10 Tag Matcher loops details 57

Table 11 Recall of the most important OneJoin parameters 59

Table 12 OneJoin FPGA resource usage 61

Table 13 OneJoin loops details 61

List of Code Snippets

Code 1 Myers' bit-parallel algorithm pseudo-code [5] 10

Code 2 Compilation line with good parameters for the `dna-little-processed.txt` dataset 19

Code 3 SYCL queue initialization example 33

Code 4 SYCL device selector example 33

Code 5 SYCL USM usage example 37

Code 6 Bitwise OR operations on integer 256 using pragma 38

Code 7 Example code for the pipeline pragma [10] 38

Code 8 Example code for the dataflow pragma [10] 39

Code 9 Command lines of the compilation flow [12] 44

Code 10 Function bitwise OR overwritten with an unroll function 49

Code 11 Code enabling selection of the appropriate queue 51

List of Appendices

Appendix 1 : Relevant links to the project 66

Introduction

In recent years, the amount of data has continued to increase to the point where, today, we have huge infrastructures dedicated to information storage. These infrastructures are the cause of enormous energy waste. However, it's difficult to eliminate them given that sectors such as education, medicine, and others depend on them. With today's technology, we are now capable of storing data in synthetic deoxyribonucleic acid (DNA). This method overcomes many of the problems that come with traditional data storage, especially knowing that all data on earth can be stored in a small test tube.

A European project called DNAMIC aims to replace these infrastructures with a mobile, self-contained, future-proof microfactory for DNA data archiving. This microfactory would only be finished after implementing the six steps that constitute it, that is, encoding, synthesis, amplification, storage, sequencing, and decoding.

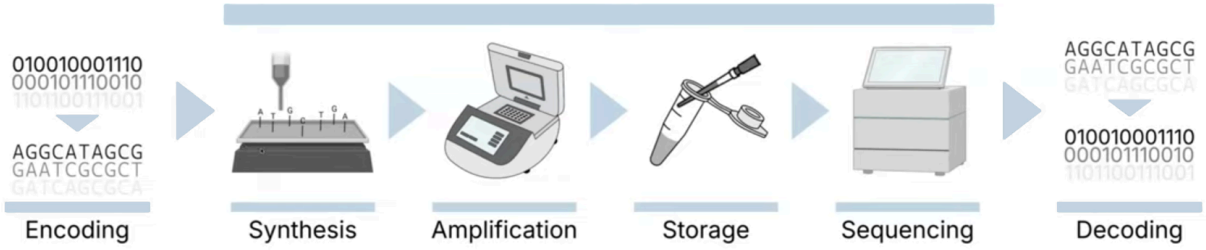


Figure 1: Diagram of the six steps constituting the micro-factory of the DNAMIC project [1]

The first goal of DNAMIC is to archive some of the data from Yareta, the archive portal for Geneva's institutions of higher learning. It provides a comprehensive solution for archiving, preserving, and sharing research data for researchers from all Geneva's higher education institutions, including the University of Geneva (UNIGE) and the University of Applied Sciences and Arts of Western Switzerland (HES-SO). Since Yareta's mission is to archive and preserve data in the long term, storing their data in the DNA would be a more sustainable and environmentally friendly solution, reducing much of the energy waste caused by their current data center.

One of many difficulties this project encounters is located at a sub-step between the sequencing and decoding steps, that we can call the sequence assembly. The reading of DNA strands is performed using a nanopore device, such as the MinION shown in Figure 2, a technology capable of reading 100 to 200 DNA strands simultaneously with an error rate of about 10 percent. In addition to those errors, DNA can also undergo mutations. These errors must then be corrected to maintain data integrity. To address this issue, engineers at DNAMIC have chosen to use the polymerase chain reaction (PCR) technique to generate multiple copies of each DNA strand, enabling error correction in subsequent steps.



Figure 2: The MinION nanopore sequencer by Oxford Nanopore Technology [2]

In addition to sequencing errors and mutations, another challenge arises from the order in which the DNA strands are read. During the reading process, the fragments can be retrieved in a shuffled or incorrect order, making it impossible to directly reconstruct the original sequence. Therefore, before error correction can be applied, the data must first be reassembled to recover the correct sequence. At this stage of the project, sequence assembly becomes essential. This seemingly small but crucial step is responsible for clustering strands originating from the same initial sequence.

DNA strand clustering is a well-known problem that generates a lot of scientific debate due to its complexity and importance in the field of DNA data storage and sequencing. To achieve

clustering, researchers draw parallels with techniques used in spell-checking tools, where differences between two text entries are calculated. This metric can be viewed as a score, where a lower value indicates a closer match. Unlike spell-checking tools, where simple differences can be sufficient, DNA strand clustering faces other challenges. Due to the variability introduced throughout the pipeline, DNA strands are not only prone to substitution errors (where one nucleotide is replaced by another) but also to deletion (where one or more nucleotides are missing) and insertion (where extra nucleotides are added). To address these alterations, more advanced distance measures must be employed, in our case, the Levenshtein distance. Today, and especially in our context, the question is no longer how to calculate this metric but how to succeed in calculating it in the shortest possible time. This project aims to support DNAMIC by exploring hardware acceleration techniques with Field-Programmable Gate Arrays (FPGAs) to optimize the calculation of the Levenshtein distance in order to efficiently manage large volumes.

The successful implementation of these improvements could have a significant impact on the DNAMIC project and the broader field of DNA data storage. By leveraging FPGA-based acceleration, large-scale DNA clustering will be feasible within practical time constraints. This advancement would enable researchers to process and classify vast datasets without compromising accuracy.

During my semester project, the goal was to find and start implementing the most relevant algorithms for this problem in Very High Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL). To achieve this, we began with an initial research phase, which allowed us to identify the most suitable algorithm for calculating the Levenshtein distance: Myers' Bit-parallel Algorithm. After fully understanding the algorithm and implementing it in C, the next objective was to implement it in VHDL as part of my bachelor's project.

However, one of the main problems with this approach was that, regardless of how fast the algorithm ran, with the number of DNA strands N, we would always need to perform $\mathrm{O}(N^2)$ comparisons. This poses a significant issue, because as the number of DNA strands N grows large, it quickly becomes impossible to carry out this many comparisons. In other words, this approach does not scale well. Therefore, another method was implemented instead: the OneJoin algorithm, which utilizes a pre-processing step to limit the number of comparisons.

This document is organized into three main parts. The first part introduces the key technical concepts needed to understand the project. The first chapter covers fundamental algorithms like the Levenshtein edit distance and Myers' Bit-parallel algorithm, while the second chapter presents the theoretical solutions for the DNAMIC challenges, including the OneJoin and the Tag Matcher approaches.

The second part focuses on the technologies used. Chapters three and four detail the FPGA hardware architecture and setup and explain the OneAPI programming framework, highlighting SYCL and the Data Parallel C++. A dedicated chapter discusses the practical usage, and the program's compilation process involving both CPU and FPGA.

The final part presents the project's implementation and performance results. Those last two chapters describe how the algorithms were applied, optimized, and analyzed. The results are discussed in detail and illustrated with graphs to clearly demonstrate the impact of the various optimizations.

Chapter 1 : The Edit Distance Algorithm

1.1 The Levenshtein Distance

In computer science, the edit distance is a metric used to measure the difference between two strings by counting the minimum number of operations required to transform one string a into another string b. This metric is commonly applied in spelling correction tools, where it helps identify and suggest the closest correct word when a misspelling is detected. Above all, the edit distance plays an important role in bioinformatics, where it is essential for comparing different DNA sequences. By doing so, it enables researchers to detect, analyze, and study mutations.

There are several types of edit distance that can be categorized according to the operations they support and allow. Some of these types are shown in Table 1.

Algorithm	**Operations Allowed**			
	Insertions	**Deletions**	**Substitutions**	**Transpositions**
Hamming Distance			✓	
Longest Common Subsequence	✓	✓		
Levenshtein Distance	✓	✓	✓	
Damerau-Levenshtein Distance	✓	✓	✓	✓
Jaro Distance				✓

Table 1: Different types of edit distance and the operations they allow [13]

As mentioned in the introduction, DNA is subject to three types of operations: insertion, deletion, and substitution. These are due to the variability introduced throughout the DNA processing pipeline. Only two of these edit distances produce the desired result, the Levenshtein distance and the Damerau-Levenshtein distance. These two differ by the fact that the Damerau-Levenshtein includes transpositions in addition to the classical edit operations. An operation that does not occur in the modifications DNA strands can undergo. Therefore, the Levenshtein distance is the most appropriate edit distance for our project.

The naive recursive approach for computing the Levenshtein distance is given by the Eq. 1 [13].

$$\operatorname{lev}(a,b) = \begin{cases} n, & \text{if } m = 0 \\ m, & \text{if } n = 0 \\ \operatorname{lev}(\operatorname{tail}(a), \operatorname{tail}(b)), & \text{if } \operatorname{head}(a) = \operatorname{head}(b) \\ 1 + \min \begin{cases} \operatorname{lev}(\operatorname{tail}(a), b), \\ \operatorname{lev}(a, \operatorname{tail}(b)), \\ \operatorname{lev}(\operatorname{tail}(a), \operatorname{tail}(b)), \end{cases} & \text{otherwise} \end{cases} \tag{1}$$

Here, n and m are the lengths of the strings a and b, respectively. The head function returns the first character of a string, while the tail function returns the remaining part of a string after removing the first character.

As an example, let's consider the following task: switching the word "party" to form the word "park". The Levenshtein distance between these two words is two, meaning it takes two operations to convert one word into the other. In this case, we substitute the "t" in "party" with a "k" and delete the "y" in "parky". The Figure 3 illustrates this process.

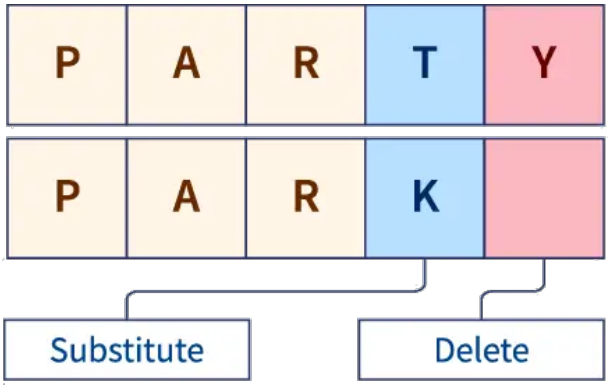


Figure 3: Representation of the Levenshtein Distance on two words [3]

1.2 Myers' Bit-parallel Algorithm

Myers' bit-parallel algorithm is an efficient method for computing the Levenshtein distance between two sequences. Unlike traditional dynamic programming approaches, which rely on filling a two-dimensional matrix, Myers' algorithm optimizes this process using bit-parallelism.

Myers' bit-parallel algorithm represents the state of the computation using bit vectors. Each bit vector compactly encodes whether a match, substitution, insertion, or deletion is needed at each position in the pattern. By leveraging bitwise operations, the algorithm efficiently simulates the dynamic programming table updates.

To explain Myers' algorithm, it is helpful to start with the Wagner-Fischer algorithm. The Wagner-Fischer algorithm is the standard dynamic programming technique for calculating the edit distance. Dynamic programming works by breaking the problem into smaller sub-problems and solving them one after the other to retrieve the solution from the original problem. In Wagner-Fischer, a matrix is used to store every Levenshtein distance between all prefixes of the strings a and b, the last computation being the distance between the two full strings.

The matrix in Figure 4 shows the result of the Wagner-Fischer algorithm applied to the words "sitting" and "kitten". The highlighted path shows the most optimized path for transforming one string into the other. If we want to correct the word "sitting" to "kitten", a diagonal movement represent substitutions (or no operation if the letters are identical), movements to the right represents insertions, and movements downward represent deletions.

		k	**i**	**t**	**t**	**e**	**n**
	0	1	2	3	4	5	6
s	1	1	2	3	4	5	6
i	2	2	1	2	3	4	5
t	3	3	2	1	2	3	4
t	4	4	3	2	1	2	3
i	5	5	4	3	2	2	3
n	6	6	5	4	3	3	2
g	7	7	6	5	4	4	3

Figure 4: Wagner-Fischer matrix between "sitting" and "kitten" [4]

One of the key strengths of the Wagner-Fischer algorithm lies in its simplicity. But, unfortunately, when dealing with long strings, this algorithm is not very efficient. With some optimization, Wagner-Fischer has a time complexity of $O(n \times k)$, n being the length of the

query (string a) and k the maximum tolerated distance [4]. Another drawback is the memory requirement, as the algorithm needs to store a matrix of size $\mathrm{O}(n \times m)$. This can be optimized to $\mathrm{O}(n)$ by storing only the current and previous columns of the matrix. Note that the memory will, in any case, become significant when comparing a lot of data simultaneously.

Now, let's look at how Myers went from computing the $n \times m$ Wagner-Fischer matrix to computing five bit vectors in parallel: Eq, Pv, Ph, Mv, and Mh, which encode the various operations. To do this, we need to understand several key principles derived from the traditional $n \times m$ matrix. In the Wagner-Fischer algorithm, when optimizing memory space, we note that a column C_j of the matrix can be computed using only the column C_{j-1} that precedes it. A related observation is that the difference between adjacent entries in any row or column is either $+1$, 0, or -1. Those two observations present a new way of approaching the problem: storing the differences between the columns instead of the values themselves. This approach originates from the 4-Russians algorithm for string comparison.

Formally, we have the *horizontal delta* $\Delta h[i, j]$ as $C[i, j] - C[i, j-1]$ and the *vertical delta* $\Delta v[i, j]$ as $C[i, j] - C[i-1, j]$ for all $(i, j) \in [1, m] \times [1, n]$, C being the dynamic programming matrix $C[0...m, 0...n]$. Δv and Δh are matrices that represent the traditional Wagner-Fischer matrix, but relative to the preceding cells, vertically or horizontally. Using several theorems, Myers manages to transform the problem of calculating the Wagner-Fischer matrix into the calculation of the matrix Δv.

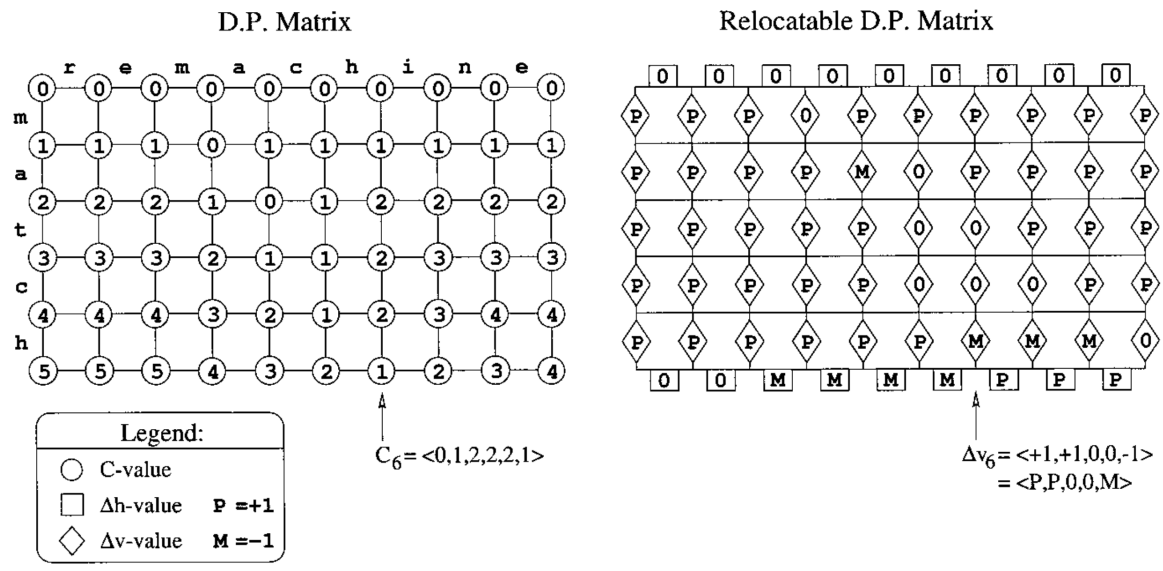


Figure 5: Myers' bit-parallel algorithm dynamic programming (d.p.) matrices [5]

To understand the origin of the bit vectors, let's look at Figure 5. On the left, the traditional matrix from the Wagner-Fischer algorithm, and on the right, the Δv matrix and the first and last rows of the Δh matrix. This illustration shows that the data in the Δv matrix are Boolean variables M or P, which indicate whether a cell's value should be increased (P) or decreased (M) by 1. They cannot both be true (be equal to 1) at the same time. To represent a row, a bit vector is created, resulting in the bit vectors Pv, Ph, Mv, and Mh, where v and h represent the axes where the operation is performed (vertical or horizontal). To correctly calculate these bit vectors, it is necessary to know whether the current characters are similar or not. The bit vector Eq is responsible for this, representing equality between characters. While Eq can be obtained through a simple comparison, it is more efficient to precompute it in $O(1)$ time for each column by storing it in a matrix $Peq[\sigma]$, where σ is the alphabet from which the two strings originate.

```
precompute Peq
Pv = 1^m
Mv = 0
score = m
for j from 1 to n:

    Eq = Peq[j]
    Xv = Eq | Mv
    Xh = (((Xv & Pv) + Pv) ^ Pv) | Xv

    if lsb(Ph) then score++
    if lsb(Mh) then score--

    Ph = Pv & Xh
    Mh = Mv | ~(Pv | Xh)
    Pv = (Mh << 1) | ~0
    Mv = (Ph << 1)
```

Code 1: Myers' bit-parallel algorithm pseudo-code [5]

Since the algorithm only contains bitwise operations, it can be easily represented using an actual electric schematic, which helps in understanding its operation. Below, the Figure 6 shows the schematic of Myers' algorithm. It is presented here primarily to give an idea of the components that the FPGA will recreate.

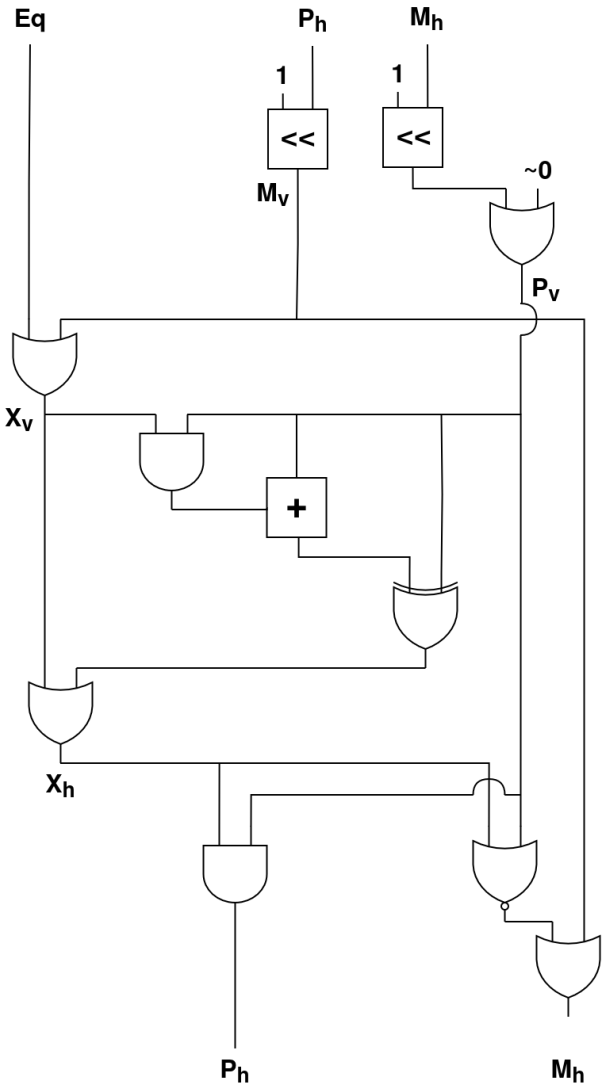


Figure 6: Schematic blocks diagram of Myers' algorithm main loop

Myers' bit-parallel algorithm has a very good time complexity of $O\left(n \times \frac{m}{w}\right)$, where w is the length of a machine word, and a space complexity of $O(n)$ [5]. Since FPGAs do not have strict limits on the length of a machine word, `w` can be set to the value of `m`, reducing the time complexity to $O(n)$. This value is one of the reasons we used this algorithm. Another reason is the use of binary operations, which simplify the implementation on FPGA. And speed up computations, as these operations can generally be performed in a single clock cycle.

Chapter 2 : Algorithms: How to Solve this Problem

2.1 Original Thoughts

The first solution considered was to compare all the DNA sequences against each other. To do this, Myers' algorithm would be used to obtain the Levenshtein distance. If it was below a certain threshold, the two DNA strands being compared would be considered identical (from the same original strand). This would group the copies together.

The problem with this approach is that, despite the speed of each comparison, it still requires performing $\frac{n \times (n-1)}{2}$ pairwise evaluations. This is not scalable, because when working with a gigabyte of data, the number of DNA strands (32 bytes) explodes to 31 million, leading to approximately 5×10^{14} comparisons.

Let's say the algorithm runs at 200Mhz, an average FPGA speed, and compute each comparison in $\mathrm{O}(n)$ using the Myers' algorithm. If one comparison takes 250 clock cycles, then processing all pairs would require around 10^{17} clock cycles. It would take $7'233$ days or about 20 years to compute all the comparisons. Even with good parallelization, it would be very difficult to achieve a reasonable time.

2.2 OneJoin

OneJoin is an edit similarity join algorithm designed to efficiently handle very large sets of sequences, such as DNA-encoded data. Unlike traditional approaches, which rely on exact calculations of the Levenshtein distance between all pairs of sequences. OneJoin relies on advanced embeddings and Locality Sensitive Hashing (LSH) techniques to drastically reduce the number of required comparisons. Its goal is to find similar strands using the Hamming distance, and not the Levenshtein distance, in order to cluster them into buckets. These buckets then contain DNA strands that are potentially close in terms of Levenshtein distance, thereby

allowing comparisons to be performed on subsets (buckets) of the dataset. OneJoin operates in three main stages: embedding, LSH, and the verification [14].

a) CGK Embedding

The appropriate edit distance for this context is the Levenshtein distance; however, its calculation is costly. The Hamming distance only counts the number of substitutions, but unlike the Levenshtein distance, it is very efficient to compute.

The CGK embedding, developed by Chakraborty et al. [15], is a method that transforms strings into a different metric space. When applied to DNA sequence datasets, the algorithm takes as input a string of length N composed of the four nucleotides A, C, G, and T. The output of the CGK embedding is a new string three times longer, with length $3N$, containing the original nucleotide characters along with multiple occurrences of a special padding character P.

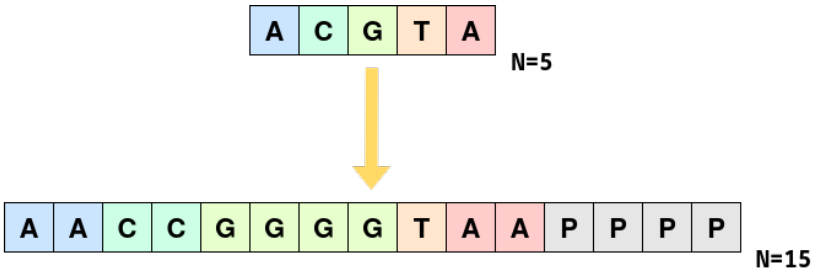


Figure 7: CGK embedding example

At each step of the process, the algorithm appends either a character from the input string or the padding character to the output. As a result, some input characters appear only once in the output, while others may be repeated randomly several times. Using random walk theory, Chakraborty et al. demonstrated that the CGK embedding guarantees that the Hamming distance between the embedded strings is bounded by the square of the edit distance between the original strings.

b) Locality Sensitive Hashing (LSH)

An important benefit of edit-to-Hamming embedding is that algorithmic tools, like Locality Sensitive Hashing (LSH), well defined for Hamming, can now be used to identify similar strings without doing an exhaustive pairwise search [16].

Locality-Sensitive Hashing (LSH) for Hamming distance is a technique that efficiently groups similar strings by mapping them to the same “buckets”. The technique relies on bit-sampling: specific bit positions are selected from each string to generate hash values. In practice, this means that two strings with a low Hamming distance, meaning they differ in only a few positions, are more likely to be hashed into the same bucket. For instance :

- Consider two 8-bit DNA sequences: $x = \mathrm{ATCGATCG}$, $y = \mathrm{ATCGATCA}$ (differs only in the last character).
- A hash function sampling bit position 7 would place both in the same bucket since bit 7 is 1 in both.

To balance precision and recall, LSH typically uses an AND-OR composition: the AND construction reduces false positives by requiring agreement on multiple sampled bits, while the OR construction reduces false negatives by considering multiple independent hashes.

The AND construction requires agreement on multiple sampled bits. For example :

- Define a hash function $H_1(x) = (\mathrm{bit}_3, \mathrm{bit}_7)$.
- x and y collide only if they match at both positions.

The OR construction uses multiple independent hashes. For example:

- Employ three hash tables: [g_1, g_2, g_3], each sampling different bits.
- Strings collide if they match in any table (g_1 OR g_2 OR g_3).

This approach allows scaling from $O(N^2)$ to near-linear comparisons, as more than 90% of dissimilar pairs are efficiently filtered out before expensive edit-distance calculations are performed. This synergy between embedding and LSH makes large-scale sequence analysis manageable.

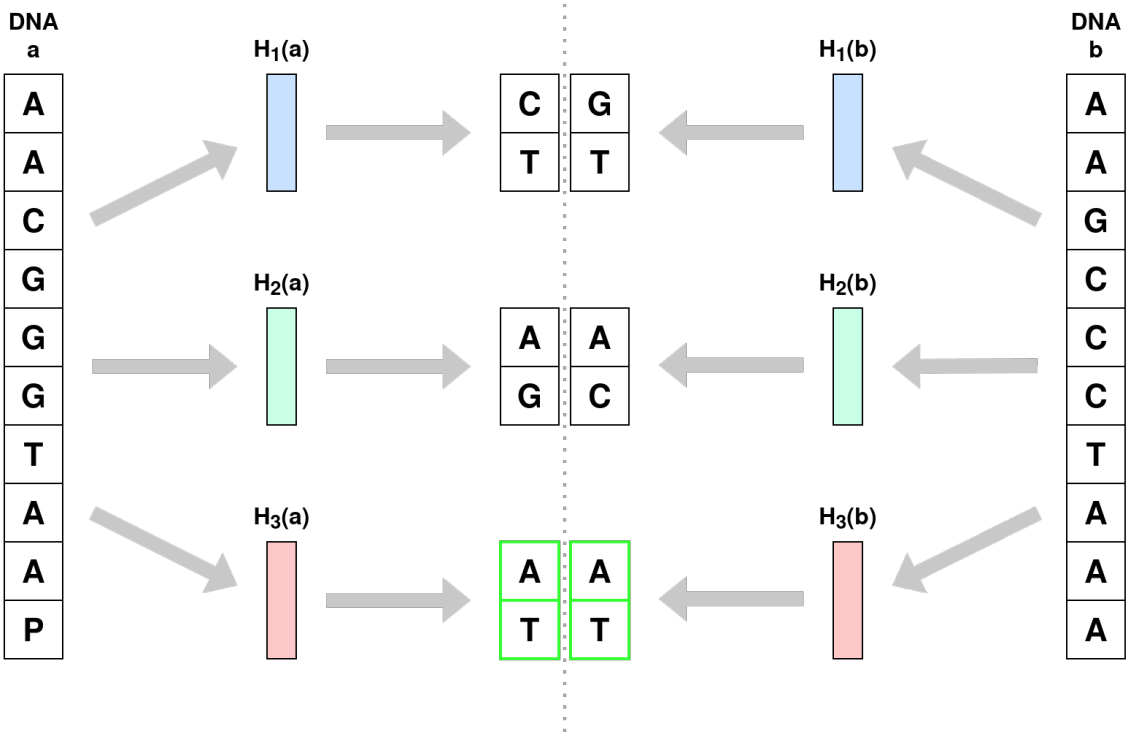


Figure 8: LSH example with two DNA strands

The Figure 8 shows how the LSH algorithm works. An embedded DNA strand is hashed multiple times (AND) giving subsequences of the strand. These subsequences are then compared to those from other strands, and if some of them are equal (OR), the algorithm identifies a match and places them in the same bucket.

c) EmbedJoin

EmbedJoin is a CPU-based, sequential algorithm designed for edit similarity joins that uses CGK embedding and locality-sensitive hashing (LSH) to minimize the number of expensive pairwise edit distance calculations.

Filtering

The filtering phase is the first and most computationally intensive stage of the algorithm. It begins by sorting the input strings according to their lengths. This step allows EmbedJoin to quickly eliminate unlikely candidate pairs by leveraging the fact that two strings with a significant length difference cannot be within the specified edit distance threshold.

Each input string is then embedded using the CGK embedding technique. The embedded strings are subsequently hashed into buckets using Locality-Sensitive Hashing (LSH) for Hamming distance. For each string, the algorithm checks all other strings in the same bucket. If the length difference between the current string and a candidate is less than the edit threshold, the pair is added to a candidate list for further verification. Otherwise, the candidate is removed from the bucket to optimize subsequent comparisons. This process is repeated for all input strings, and duplicate pairs are eliminated to avoid redundant computations.

Verification

After the filtering phase, EmbedJoin has generated a candidate list of string pairs that are likely to be within the specified edit distance. The verification phase then computes the exact edit distance for each candidate pair. This step is much less computationally demanding than filtering, as the number of candidate pairs is typically only a small fraction of all possible pairs due to the effectiveness of the embedding and LSH-based filtering.

For each candidate pair, EmbedJoin employs a standard dynamic programming algorithm to compute the distance of Levenshtein. If the computed distance is below the user-defined threshold, the pair is included in the final output buckets. The verification phase is executed in a straightforward, sequential manner, with each thread or process handling a subset of the candidate list.

d) Parameters

OneJoin effectiveness depends heavily on several configurable parameters that influence key tradeoffs between performance, accuracy, and coverage. Table 2 summarizes these parameters, their effects on critical metrics, and their default values from the OneJoin paper.

Parameter	Symbol	Brief description
NUM_STR	r	independent CGK embeddings per read
NUM_HASH	z	hash functions (OR-construction) per embedding
NUM_BITS	m	bits sampled inside each hash (AND-construction)
NUM_CHAR		alphabet size of the dataset
K_INPUT	K	edit-distance threshold passed to the verifier
SHIFT		edit-band stride used when slicing strings during embedding
NUM_REP	$\lceil \frac{K}{\text{SHIFT}} \rceil$	replicated sub-strings embedded per read
samplingrange	s	highest character position inspected by LSH; determines embedded length
countfilter	c	min occurrences of the same candidate pair before verification
batch_size	B	reads processed together; limited by GPU / RAM

Table 2: Description of the OneJoin algorithm parameters

Table 3 shows their effects on critical metrics. It serves as a reference for tailoring OneJoin to specific datasets, balancing speed and reliability according to application needs (e.g., large-scale DNA storage).

Parameter	Speed impact	Recall impact	Precision impact	False positives impact	True positives impact
NUM_STR	↑	↑	↓	↑	↑
NUM_HASH	↑	↑	↓	↑	↑
NUM_BITS	↑	↓	↑	↓	↓
NUM_CHAR	↔	↔	↔	↔	↔
K_INPUT	↑	↑	↓	↑	↑
SHIFT	↓	↑	↔	↔	↑
NUM_REP	↑	↑	↔	↔	↑
samplingrange	↑	↑	↔	↔	↑
countfilter	↓	↓	↑	↓	↓
batch_size	↑ or ↓	↔	↔	↔	↔

Table 3: Impacts of parameters on algorithm performance and quality

The choice of parameters depends largely on the dataset. In this project, the dataset-generator script was used to create multiple datasets containing mutated DNA strands. The dataset used for algorithm testing was a relatively small set of 7′500 DNA strands. The details of this dataset are presented in Table 4.

Parameter	dna-little-processed.txt
Clusters (-n)	250
Strands / cluster (-c)	30
Total strands	7′500
Length strands	207
Avg. error %	10.32
Substitution %	4.63
Deletion %	4.22
Insertion %	1.47

Table 4: Sequencing parameters for `dna-little-processed.txt`

According to the parameters of this dataset, several parameters of the algorithm were defined, such as NUM_CHAR and K. The choice of K is critical:

– If K is too small, true duplicates may be missed.

– If K is too large, the risk of accepting false positives increases.

For our 207-nucleotide strands, the error rate is around 10% ($\sim$ 21 characters). On average, the edit distance between two reads is therefore about 42. Adding a small margin of 6 led us to choose $K = 48$, which provided good results in practice.

The remaining parameters were determined empirically, through trial-and-error, based on their impact on the final results after several tests. The compilation command used for the experiments is given in Code 2.

```
make update PARAMS="-DDEF_NUM_STR=24 -DDEF_NUM_HASH=28 -DDEF_NUM_BITS=11 -
DDEF_NUM_CHAR=4 -DDEF_K_INPUT=48 -DDEF_SHIFT=16"
./onejoin -r test/dna-little-processed.txt -a 2 -d 1 -s 300 -c 1 -b 30
```

Code 2: Compilation line with good parameters for the `dna-little-processed.txt` dataset

Note : It is worth mentioning that some tests were also performed on larger datasets. However, when using the original *oneoligo* code, these attempts systematically resulted in one of the following outcomes, suggesting that the work should be limited to the `dna-little-processed` dataset at this stage :

– Extremely low coverage

– No buckets generated

– System crash, potentially due to an out-of-memory condition

This is a known issue, and the authors have been contacted.

2.3 Tag Matcher

Despite the acceleration provided by OneJoin, it was observed during the project that it is not fast enough when it comes to clustering large volumes of data. An idea emerged during discussions with the DNAMIC team at UNIGE: namely, to add a large pre-filtering step to the data, in order

to divide it into several groups, each containing up to 65′280 distinct strands. These groups would generally contain data originating from the same file, archived in DNA format.

To achieve this pre-division, the technique involves adding tags at the beginning of the DNA strands. These tags must be sufficiently separated in terms of Levenshtein distance so that they can still be identified even in the presence of DNA sequencing errors. The creation of these tags represented a significant effort by one of the UNIGE collaborators.

To cluster the data, the first step is to isolate the tag. In practice, this step is fairly complex due to the characteristic of the Levenshtein distance. Given the time constraints of the project, it was difficult to address it comprehensively. To bypass this step, we used a list of noisy tags generated using a Python script. This script is designed in accordance with the error rate that can be encountered in nanopore DNA sequencing.

The second and final step involves matching each noisy tag to its reference tag, thereby creating a tag pair. This comparison is performed between the noisy tag and all reference tags using a SYCL implementation of the Myers' algorithm. During this process, it is possible for a noisy tag to be equidistant from several reference tags. To avoid losing information in such cases, all equidistant reference tags are considered as potential candidates. Several pairs are thus created and forwarded to the next processing stage. Incorrect pairs are generally filtered out later by the OneJoin algorithm.

The final pipeline, combining the Tag Matcher and OneJoin, is illustrated in Figure 9. Note that the clustering and data recovery steps are not discussed in this project.

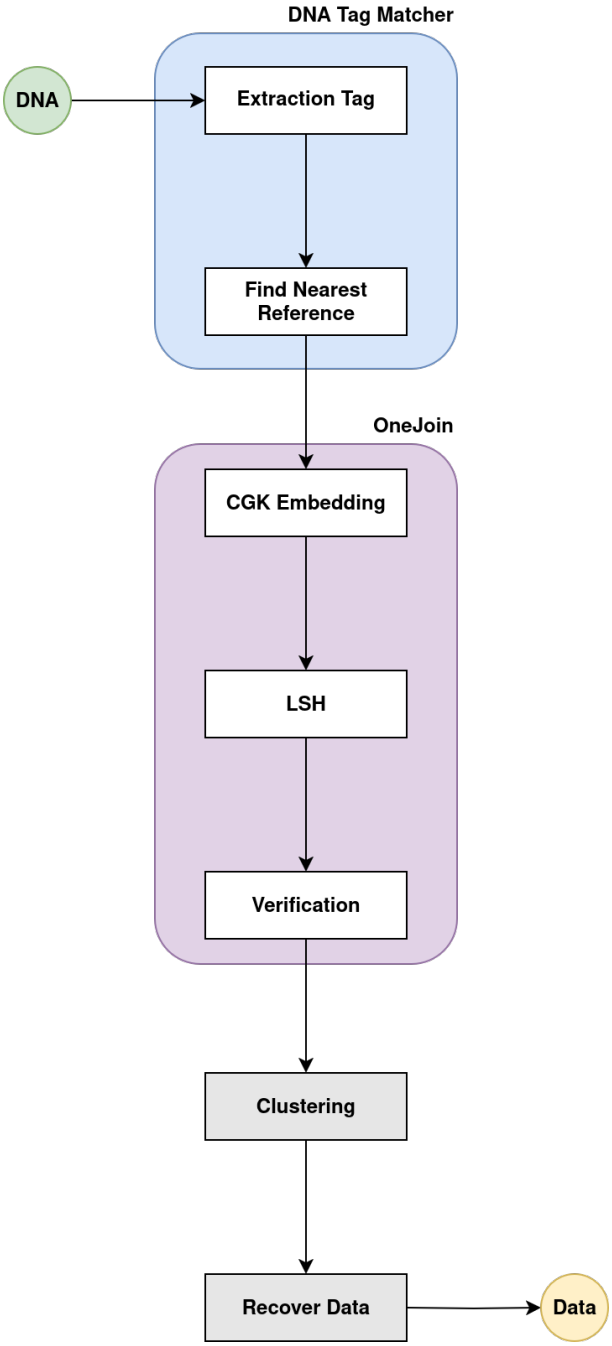


Figure 9: Tag Matcher algorithm pipeline

Chapter 3 : FPGA: Hardware Architecture and Programming

3.1 The Agilex 7 Starter Kit

The Agilex 7 Starter Kit, abbreviated as A7SK, is a development board created by Terasic. It leverages the latest Intel (formerly Altera) Agilex 7 SoC FPGA F-Series device, featuring 2.7 million logic elements. It comes with a Hard Processor System (HPS) ARM Cortex-A53 quad-core 64-bits working up to 1,5GHz [6]. The board offers multiple interfaces, including PCI Express 4.0 x8 and up to 16GB of DDR4 memory, to provide optimal acceleration. In top of that, it provides 259Mb of M20K and 28Mb of MLAB memory [17]. Additionally, the board includes an FMC+ expansion connector, ensuring compatibility with a variety of daughter cards for extended functionality.

The Agilex 7 FPGA fabric delivers 50% higher performance and 40% lower power consumption compared to previous generation devices, making it ideal for demanding workloads. The board also supports Intel's oneAPI and OpenCL frameworks, allowing developers to easily customize hardware acceleration workloads. Figure 10 shows a block diagram of all the peripherals and connections of the A7SK :

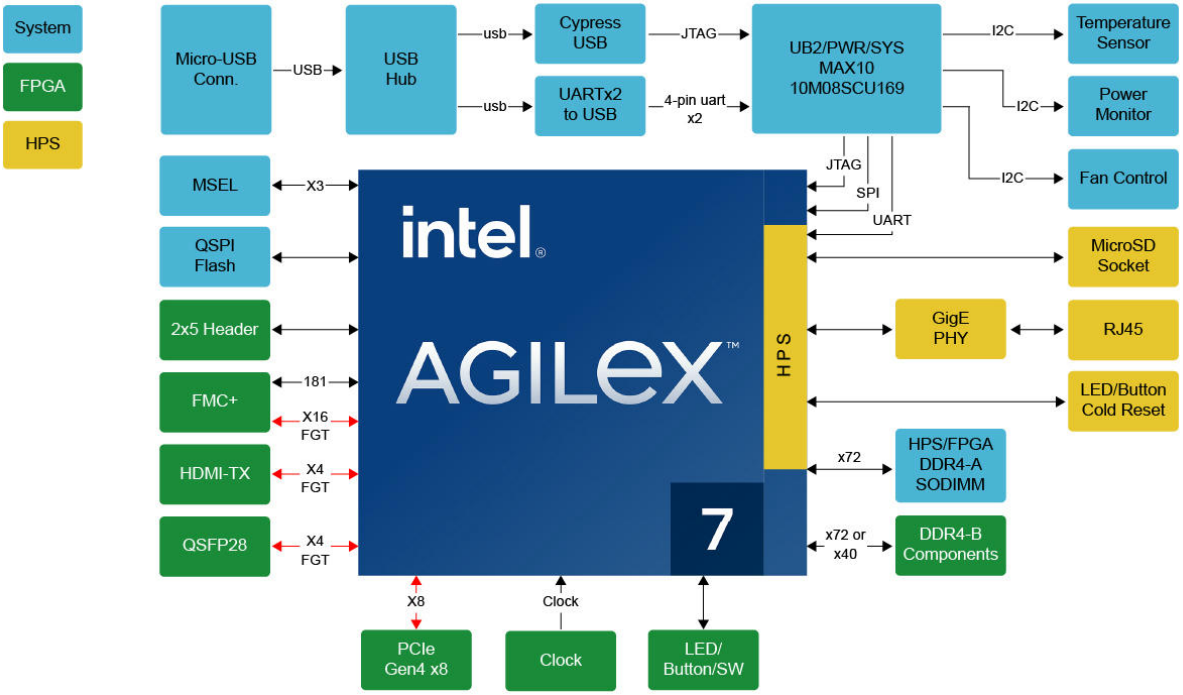


Figure 10: Block diagram of the Agilex 7 Starter Kits [6]

3.2 Open FPGA Stack (OFS)

Open FPGA Stack (OFS) is a fully open-source hardware and software infrastructure that is transforming the development of custom FPGA platforms. It consists of multiple components that enable rapid FPGA development and is available through their Git repositories. OFS is utilized by Intel (formerly Altera) as well as by several third-party platforms, supporting Intel FPGA families such as Stratix® 10, Agilex™ FPGA, and future devices.

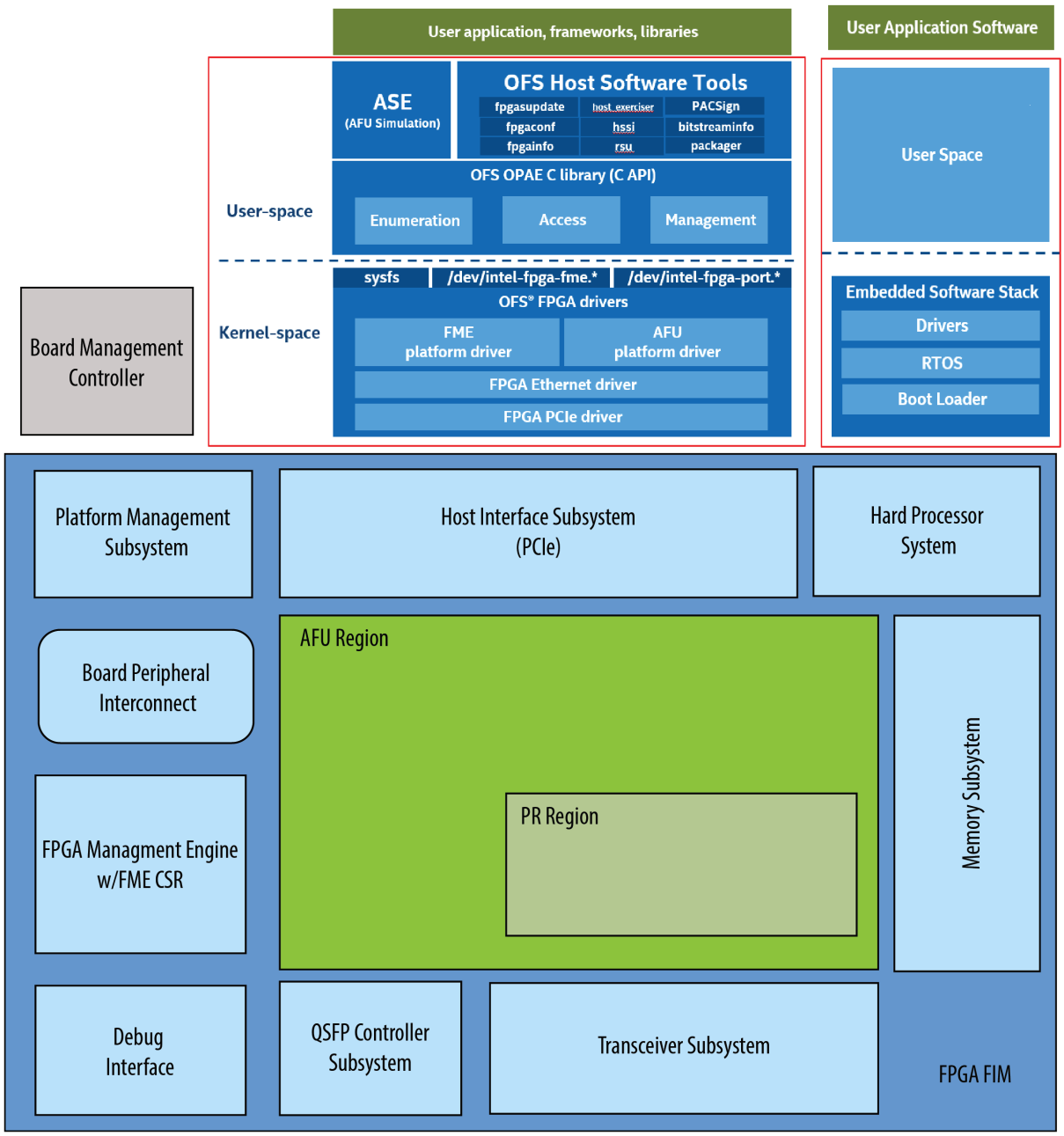


Figure 11: Block diagram of the FPGA Interface Manager (FIM) [7]

a) Hardware Infrastructure

The FPGA Interface Manager (FIM) acts as the foundational "shell" that provides platform management, functionality, clocks, resets, and standard interfaces to both the host system and the Accelerator Functional Unit (AFU). Think of the FIM as the operating system for the FPGA, it handles all the low-level communication protocols and platform-specific details so that

developers can focus on their actual application logic [7]. These reference FIMs are provided as working examples to help users develop their own custom platforms.

The Accelerator Functional Unit (AFU) is the dedicated region where custom workload implementations are deployed. This is where the OneJoin algorithm would be implemented. The AFU includes partial reconfiguration (PR) regions for dynamic programming, allowing the algorithm implementation to be updated without reconfiguring the entire FPGA.

Essentially, the AFU is the computational heart of the FPGA design, performing the actual data processing work, while the FIM handles all the platform-specific interfaces and communication protocols.

The Board Management Controller (BMC) is responsible for controlling, monitoring, and granting access to board features such as power management, thermal monitoring, and more. It provides a root of trust and conducts authentication to verify that bitstreams come from a trusted source.

The oneAPI Accelerator Support Package (ASP) is a collection of hardware and software components enabling High-Level Design (HLD) workload support. HLD abstracts the FPGA hardware and design flow, allowing users to create custom FPGA custom hardware using familiar programming languages like SYCL and development environments.

This is particularly relevant for the project since it uses SYCL to implement the algorithms discussed earlier. The ASP provides the necessary abstraction layer between the SYCL code and the underlying FPGA hardware, allowing the user to focus on the algorithm implementation rather than low-level hardware details.

b) Software Infrastructure

The open-source kernel drivers provide a layer that decomposes the implemented functionality into sets of individual device features. These drivers handle the low-level communication

between the host system and the FPGA hardware. The DFL kernel drivers represent a specialized layer of the Linux kernel. They define a structured list of feature headers within the device's MMIO (memory-mapped input/output) space, creating an extensible way of adding and discovering FPGA features [7].

The Open Programmable Acceleration Engine (OPAE) libraries and APIs are lightweight, user-space libraries that provide abstraction for FPGA resources. OPAE installs tools to expedite development, providing useful functions such as displaying FPGA information, implementing secure firmware updates, enabling virtualization, downloading new images to the board, and more. These tools are essential for debugging and monitoring the implementation of the project.

3.3 Setup

While working on the project, I encountered many errors caused by incompatible versions of different tools. During an initial attempt on my base distro, CachyOS, I was unable to correctly configure the environment. One of the several errors I encountered was related to the compatibility between libc and gcc, which I was unable to resolve. A second attempt was made on Ubuntu 22.04. A distribution said to be compatible with all the tools, yet when using the A7SK board, it did not respond. This was likely due to an error related to a poorly configured kernel. Finally, the configuration I used was very close to the one recommended by OFS for oneAPI on Agilex 7 series F FPGAs [7]. Table 5 summarizes the configurations.

Component / Tool	Version
FPGA Platform	Agilex® 7 FPGA F-Series Development Kit (2x F-Tile)
Operating System	RHEL 8.8 , Kernel : 4.18.0-477
Linux Kernel Driver	Tag: intel-1.11.0-2
OPAE	Branch: release/2.13.0, Tag: 2.13.0-3
OFS	Tag: ofs-2024.2-1
ASP	oneapi-asp-a7sk
Quartus Prime Pro Edition	Version 23.2 Pro Edition
Intel® oneAPI Base Toolkit	2025.0.1
FPGA Support Package	2025.0.1

Table 5: Versions of components and tools used for the FPGA development

These versions were specifically chosen to be compatible with FPGA. Notably, the oneAPI Base Toolkit is at version 2025.2.0 but no longer includes FPGA support since version 2025.1.0. Additionally, certain hardware features must be present on the PC using the configuration. Such as Single Root I/O Virtualization (SR-IOV), a technology that allows a PCI Express device, like an FPGA, to be shared among multiple virtual machines (VMs) while maintaining near-native hardware performance. For Intel CPUs, this feature is available on 8th generation or newer CPUs, which was not my case and forced me to completely change machines. Another feature to enable in the BIOS is the IOMMU (Input-Output Memory Management Unit), which ensures secure and efficient management of direct memory access (DMA) between PCIe devices and system memory.

Chapter 4 : OneAPI

Traditionally, programming FPGAs relies on hardware description languages like Verilog HDL or VHDL, which require specifying low-level circuit structures and functionalities. While powerful, these languages are complex and challenging for most developers, leading to high development costs and steep learning curves. High-Level Synthesis (HLS) has emerged as a solution, allowing developers to describe hardware behavior using more familiar languages like C or C++. HLS tools then translate this high-level code into register-transfer level designs, simplifying hardware development.

Initially, the goal of this project was to implement an algorithm (e.g. The Myers' Bit-parallel) entirely in VHDL. However, maintaining and transferring this low-level code proved challenging for the client, so we decided to shift to an HLS-based approach using oneAPI, making the project more accessible and manageable for future development.

4.1 OneAPI Overview

CPUs, GPUs, and FPGAs are all essential computing engines, but each excels at different types of workloads due to architectural differences. As modern applications become more diverse, achieving optimal performance increasingly requires a mix of specialized architectures, including CPUs, GPUs, AI accelerators, and FPGAs. However, programming each of these devices typically involves different languages and tools, making code less portable and increasing software development effort.

To address these challenges, Intel introduced oneAPI, a unified programming model designed to simplify development across CPUs, GPUs, FPGAs, and other accelerators. By providing a single set of tools, languages, and libraries, oneAPI enables developers to write code once and deploy it efficiently across diverse hardware platforms, streamlining workflow and enhancing

code portability. Those features are offered thanks to the use of modern C++ features and the help of SYCL to express parallelism.

Within the oneAPI ecosystem, Intel provides FPGA programming support through tools like the Intel FPGA SDK for OpenCL and the Intel FPGA Add-on for the oneAPI Base Toolkit. These tools allow developers to program FPGAs using the DPC++ (Data Parallel C++) compiler. Once development is complete, the DPC++, in combination with Intel Quartus software and the relevant board support package, efficiently compiles and deploys the code onto the FPGA hardware. Figure 12 illustrates some of the design layers between oneAPI and the Open FPGA Stack.

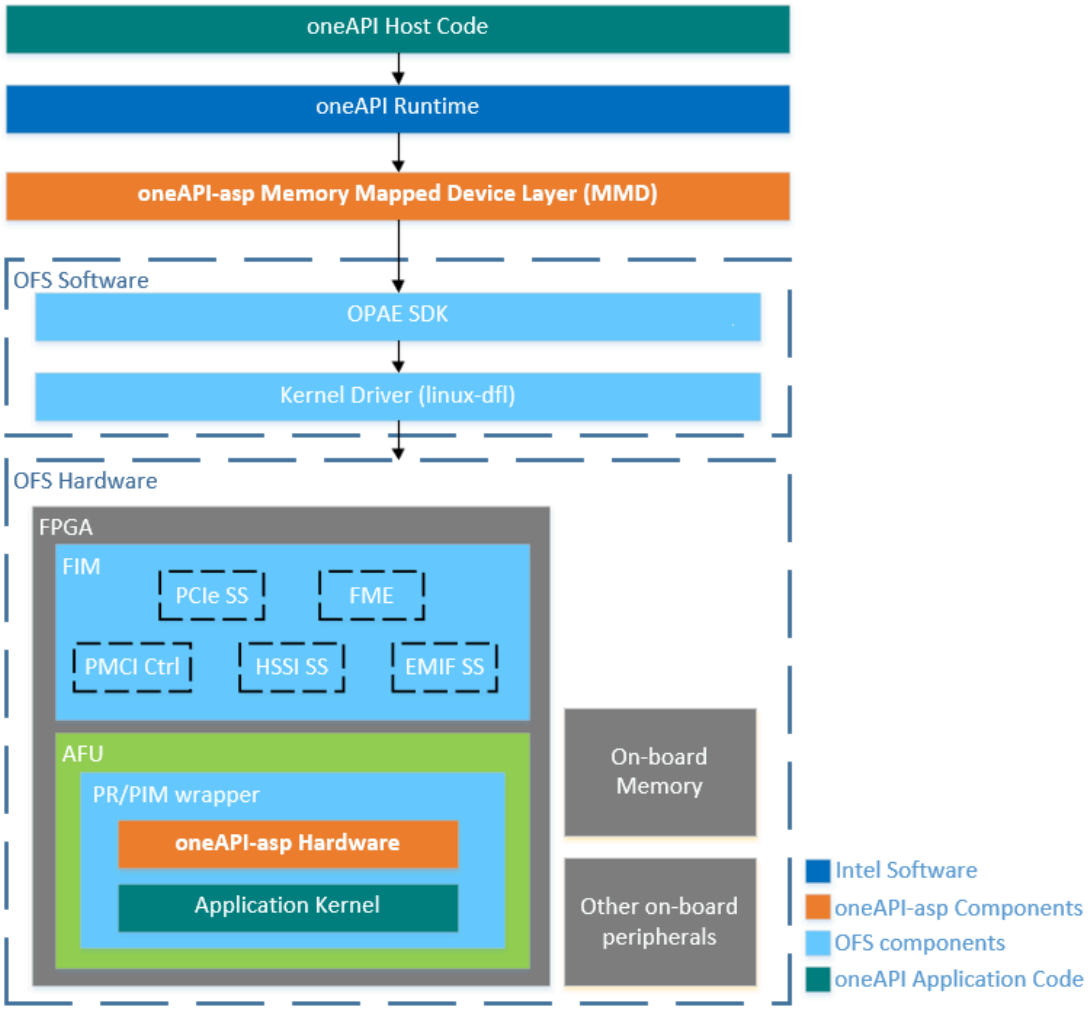


Figure 12: High level design tool on OFS platforms [7]

4.2 SYCL and Data Parallel C++ (DPC++) Overview

SYCL (pronounced “sick-el”) is an open standard developed by the Khronos Group for programming heterogeneous architectures using standard C++. It provides a single-source programming model, allowing both host and device code to be written in the same file called a “big binary”. SYCL is designed to enable efficient execution on a wide range of hardware, including CPUs, GPUs, and FPGAs, without requiring platform-specific code changes.



Figure 13: SYCL logo

SYCL works by expressing parallelism through kernels, which are functions executed on the device (such as a GPU or FPGA). The runtime manages data transfer between the host and device, as well as the scheduling of kernels. SYCL uses modern C++ features, such as templates and lambdas, to define kernels and data management abstractions. The programmer writes a single codebase, and SYCL ensures that the appropriate hardware is targeted at runtime, depending on the available devices and configuration.

The Intel® oneAPI DPC++/C++ Compiler is Intel’s implementation of the SYCL standard. It optimizes applications for Intel® 64 architectures on Linux and Windows, supporting the latest C and C++ standards. It builds upon SYCL by introducing additional extensions and features aimed at optimizing performance and increasing productivity for developers targeting Intel architectures. The DPC++/C++ Compiler is included as part of the oneAPI framework.

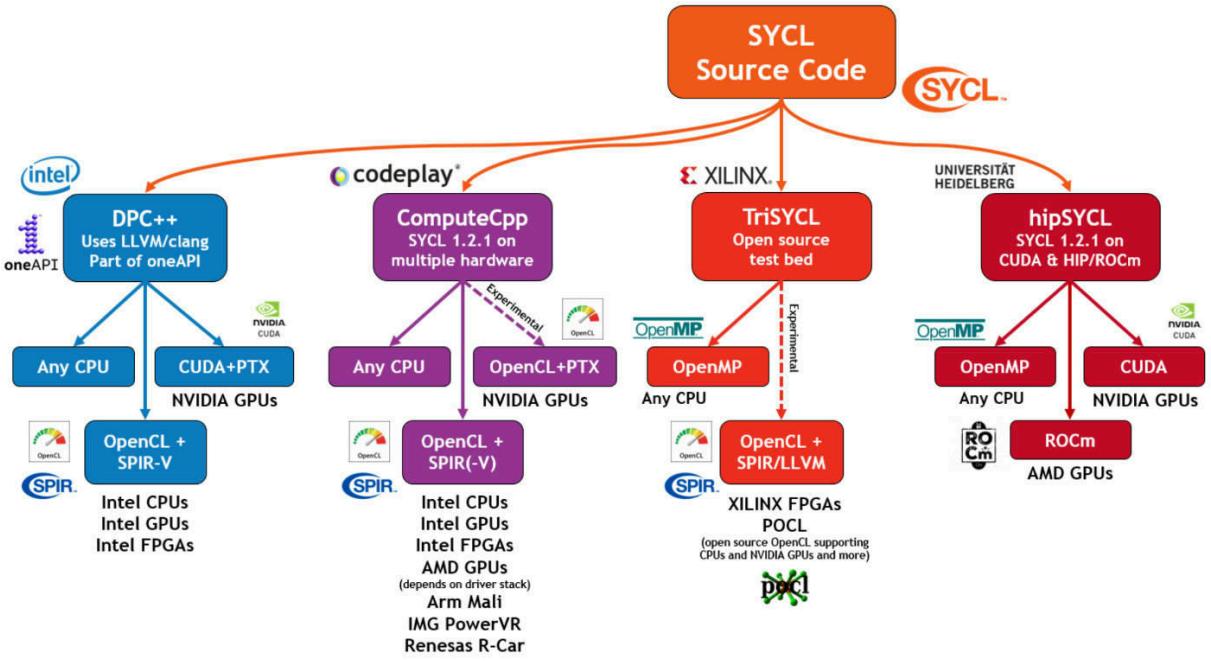


Figure 14: SYCL implementations and their supported hardware backends [8]

4.3 Advantages of SYCL for FPGA Acceleration

SYCL offers significant benefits for developers seeking to leverage FPGA acceleration in their applications. One of the primary advantages is hardware abstraction: SYCL enables the writing of code that can be efficiently compiled for FPGA using High-Level Synthesis (HLS) tools, eliminating the need for FPGA-specific programming. This abstraction simplifies the development process and broadens the accessibility of FPGA technology. Notably, SYCL is based on C++, making it much easier for software developers to target FPGAs without having to learn traditional HDLs. This lowers the entry barrier and accelerates the adoption of FPGA acceleration, especially for those with a software background.

Another key benefit is performance portability. With SYCL, it is possible to target specific kernels for FPGA execution while running other parts of an application on CPU or GPU. This flexibility allows developers to optimize for both performance and energy efficiency, depending on the requirements of individual computational tasks.

SYCL's single-source approach further simplifies development. Developers can experiment with FPGA acceleration without the need to rewrite their entire codebase, making it easier to integrate FPGA into existing pipelines. This approach streamlines the adoption of FPGA technology and reduces development overhead, something that was searched by the DNAMIC engineers.

4.4 Programming with DPC++

Since DPC++ is based on the C++ programming language, its syntax and usage are very similar to standard C++. The differences arise when programming for different types of devices. Figure 15 is a first example of SYCL code, with its various elements explained in the following sections.

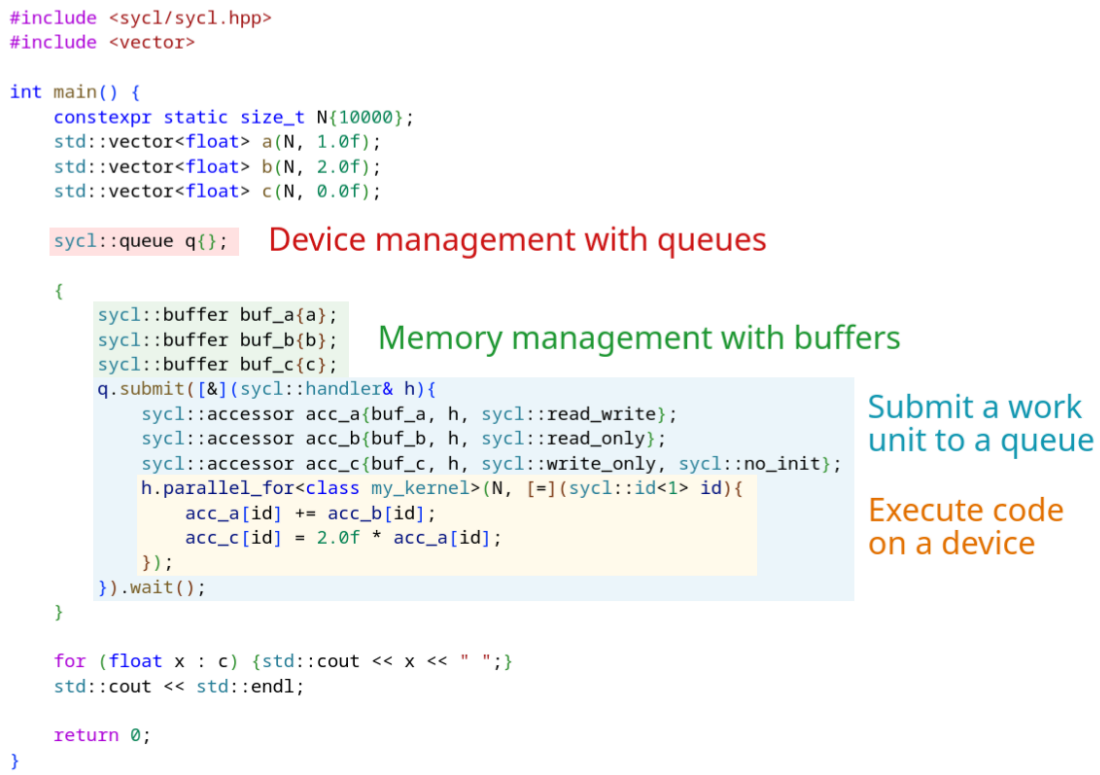


Figure 15: A first example of SYCL code explained [9]

a) Queues and Device Selector

In DPC++, the concept of device code revolves around the use of kernels. A kernel is a piece of code that is sent to a device such as a CPU, GPU, or FPGA. It is at the moment of sending this code that queues come into play.

```
sycl::queue q();

q.submit([&](sycl::handler& h){
    // Command group function
}).wait();
```

Code 3: SYCL queue initialization example

A queue has an associated device, and any commands enqueued to it will target that device. The SYCL runtime discovers the set of devices and their platforms that are available in the system.

To simplify the process of traversing the system topology, SYCL provides device selectors. A device selector is a callable C++ object that defines a heuristic for scoring devices. A device selector takes a parameter of type `const device` and assigns it a "score". It is used to query all devices and return the one with the highest "score". SYCL provides a number of standard device selectors, such as `default_selector_v`, `gpu_selector_v`, etc.

```
int my_gpu_selector(const device& dev) {
    if (dev.is_gpu()){
        return 1;
    }
    else {
        return -1;
    }
}

int main(int argc, char *argv[]) {
  auto gpuQueue = queue{my_gpu_selector};
}
```

Code 4: SYCL device selector example

b) Parallel For

A key feature of SYCL for enabling efficient parallel processing on heterogeneous hardware such as FPGA, CPU, and GPU is the `parallel_for` construct. This construct allows developers to express data-parallel computations in a straightforward manner, where the same kernel function is executed concurrently across a range of data elements. In the context of DNA sequence analysis or similarity joins, `parallel_for` enables the simultaneous processing of multiple sequences, significantly increasing throughput and reducing overall execution time.

When targeting FPGA with DPC++, the `parallel_for` construct is mapped to hardware pipelines by the High-Level Synthesis (HLS) tools. This mapping allows FPGA resources to be used efficiently, exploiting spatial parallelism where independent operations are performed simultaneously on different data streams. Additionally, SYCL's memory management features, such as buffers and accessors, ensure that data is automatically transferred between host and device memory, further simplifying the integration of `parallel_for` into complex pipelines. Figure 15 illustrates how this construct is used.

c) Memory Management

Memory management is essential for accelerating applications on FPGAs and other heterogeneous hardware. SYCL simplifies this by providing high-level abstractions for data movement and access, allowing data to be transferred efficiently between the CPU and FPGA without requiring low-level code from the developer. In SYCL there are two models for managing data.

SYCL's buffer and accessor model enables seamless data sharing between host and device. Buffers manage data on the host, while the SYCL runtime automatically handles transfers to and from the device. When a kernel is launched, the entire runtime ensures that all required data is available on the device and synchronizes the results back to the host. The example in Figure 15 demonstrates memory management using buffers.

There are multiple ways to initialize an accessor, depending on how the data need to be accessed. An accessor is always associated with a command group, leading to the connection shown in Figure 16.

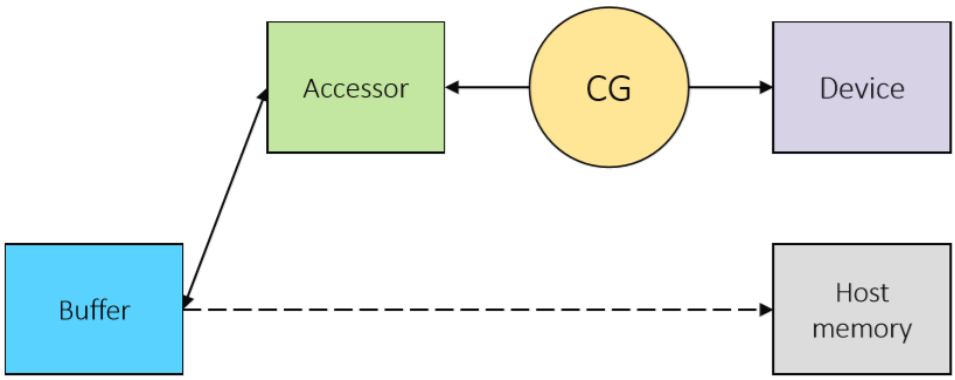


Figure 16: Schema of a buffer access memory [9]

The buffer being accessed is connected to the accessor based on how the data is used. The accessor is linked to the command group that uses it and, finally, to the device.

Unified shared memory (USM) provides an alternative pointer-based data management model to the accessor-buffer model. USM memory allocations return pointers that are consistent between the host application and kernel functions on a device. Its pointer-based API is more familiar to C or C++ programmers.

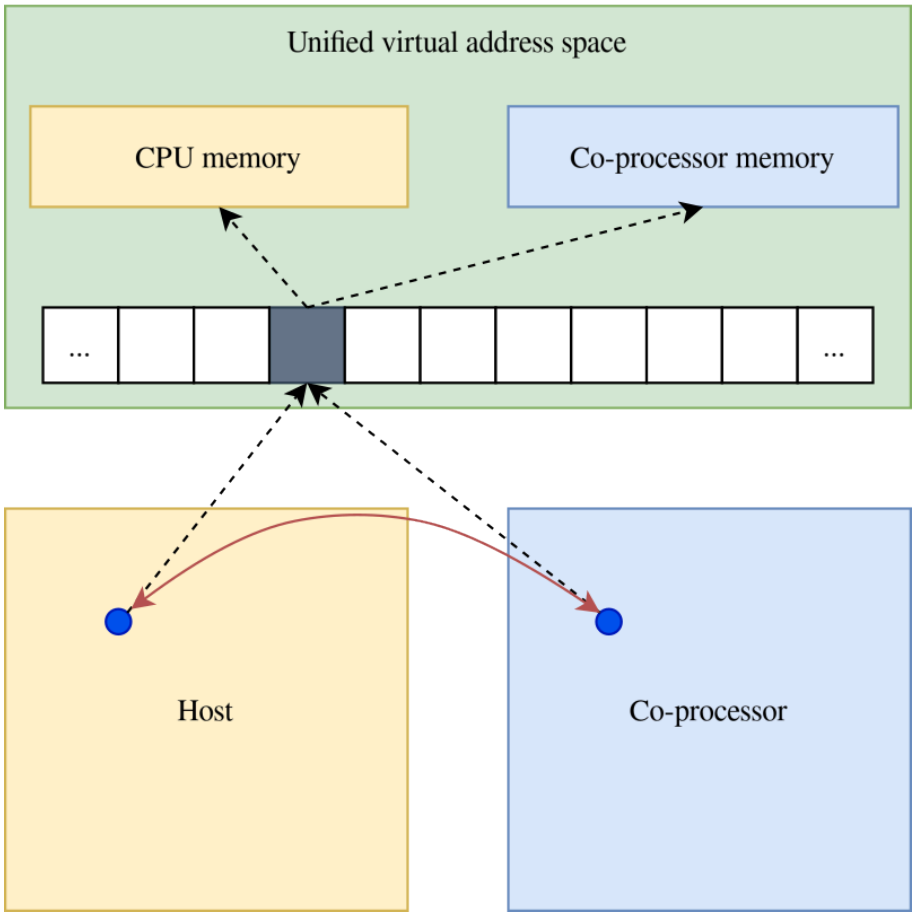


Figure 17: Host CPU and co-processor sharing a single address space for USM allocations [9]

Figure 17 illustrates how the host CPU and a co-processor (the SYCL device such as a GPU or FPGA) share a single unified virtual address space. This allow pointers to USM allocations to reference the same memory addresses on both sides. USM can be allocated in three different ways : host, device and shared.

Type	Description	Access Host	Access Device	Nominal Location
device	device global allocations	X	✓	device
host	host allocation	✓	✓	host
shared	allocations shared between both	✓	✓	migrates as required

Table 6: USM allocation types

Here is a basic example of the use of USM :

```
float a[size], b[size], r[size];
// init a, b and r with values...

auto q = sycl::queue{usm_selector};

auto ptrA = sycl::malloc_device<float>(size, q);
auto ptrB = sycl::malloc_device<float>(size, q);
auto ptrR = sycl::malloc_device<float>(size, q);

usmQueue.memcpy(ptrA, a, sizeof(float) * size).wait();
usmQueue.memcpy(ptrB, b, sizeof(float) * size).wait();

q.parallel_for<vector_add>(sycl::range{size}, [=](sycl::id<1> idx) {
    auto g_id = idx[0];
    ptrR[g_id] = ptrA[g_id] + ptrB[g_id];
}).wait();

q.memcpy(r, ptrR, sizeof(float) * size).wait();

sycl::free(ptrA, q);
sycl::free(ptrB, q);
sycl::free(ptrR, q);
```

Code 5: SYCL USM usage example

d) Pragma

In DPC++ programming for FPGAs, pragma directives are special hints given to the compiler used to optimize the hardware implementation of the code. They help control how loops and other code structures are translated into FPGA hardware, allowing for better performance and more efficient use of resources.

A common pragma is `#pragma unroll`, which tells the compiler to fully or partially unroll a loop [18]. Loop unrolling can increase parallelism by generating multiple hardware operations for each loop iteration. For example:

```
DNABitVector256 operator|(const DNABitVector256& other) const {
    DNABitVector256 result;

    #pragma unroll
    for (int i = 0; i < NUM_WORDS; i++) {
        result.words[i] = words[i] | other.words[i];
    }

    return result;
}
```

Code 6: Bitwise OR operations on integer 256 using pragma

This instructs the compiler to replicate the loop body multiple times, effectively executing iterations in parallel on the FPGA. Using pragmas strategically is essential for tuning an FPGA design to meet performance goals without wasting resources.

A second pragma useful for this type of project is the pipeline pragma. This pragma is responsible for reducing the initiation interval (II) for a function or loop by allowing multiple operations to run at the same time. Let's say we have a function `func` doing three operations: `op_read`, `op_compute` and `op_write`. Below is an example of how it would be with and without the pipeline.

```
void func(m, n, o) {
  for (i = 2; i >= 0; i--) {
    op_read;
    op_compute;
    op_write;
  }
}
```

Code 7: Example code for the pipeline pragma [10]

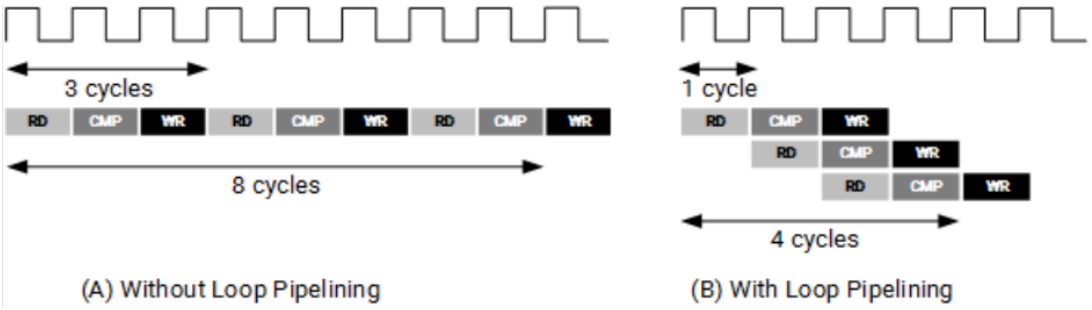


Figure 18: Schema of an example code being executed with and without pipeline [10]

In Figure 18, (A) shows the execution timeline of the `func` function without the pipeline, pragma where three clock cycles separate each loop (`II=3`). Using the pragma, (B) shows that the clock cycles are now reduced to one (`II=1`). To instruct the oneAPI DPC++/C++ Compiler to attempt loop pipelining, the flag `-Xsauto-pipeline` must be added at the compilation. Another way to do it, is by using the `[[intel::initiation_interval(n)]]` attribute to instruct the compiler to attempt to set the initiation interval to `n`.

The dataflow pragma implements task-level pipelining by enabling functions and loops to execute concurrently. Thereby increasing the parallelism in the RTL implementation and boosting overall design throughput.

```
void top(a, b, c, d) {
  for (i = 2; i >= 0; i--) {
    func_A(a, b, i1);
    func_B(c, i1, i2);
    func_C(d, i2);
  }
}
```

Code 8: Example code for the dataflow pragma [10]

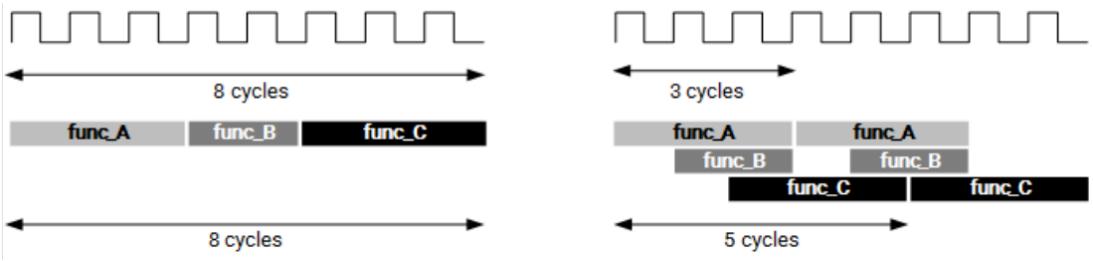


Figure 19: Schema of an example code being executed with and without dataflow [10]

Figure 19 shows the difference between the execution of Code 8 with (B) and without (A) the dataflow pragma. When the dataflow pragma is applied, the HLS tool examines the flow of data between sequential functions or loops and introduces channels (implemented with ping-pong RAMs or FIFOs). These channels enable consumer functions or loops to begin execution before the corresponding producer functions or loops have fully completed. Here, it saves up to three clock cycles per loop. It's important to note that the data dependencies between functions can limit this optimization. For instance, if `func_B` requires the output from `func_A`, `func_B` cannot begin execution until `func_A` has completed, introducing a two-clock-cycle delay per loop iteration. This mechanism creates latency between functions but ensures that functions or loops only access data after it has been properly processed.

To take advantage of this pragma with the oneAPI DPC++/C++ Compiler, it is not a matter of simply adding a flag or a `#pragma` before a loop, but rather of designing the code in a way that avoids unnecessary data transfers. The use of memory buffers/accessors also helps in this regard, as they provide automated and optimized memory access.

Chapter 5 : Heterogeneous Programming with CPU and FPGA

5.1 Compiling for Mixed Devices

When creating a program with offload, the compiler must generate code for both the host and the device. This requires two compilation: one for the host code and one for the device code.

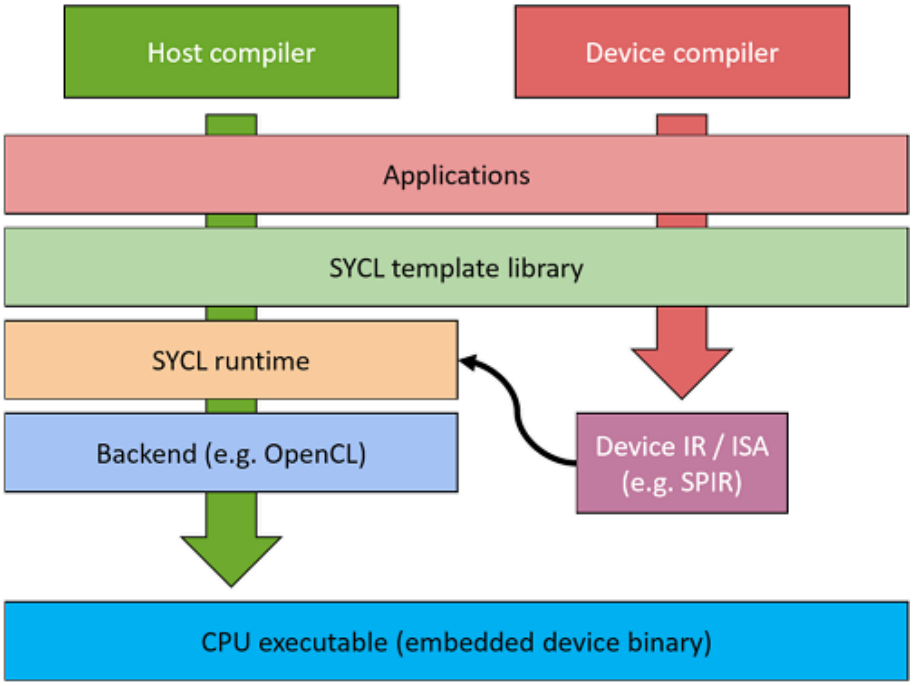


Figure 20: Basic compilation flow [9]

Thus, it is necessary to have the appropriate backend for the device being programmed. SYCL currently supports backends such as OpenCL, CUDA, HIP, OpenMP, and others.

When writing code for multiple devices, it is important to ensure proper compilation, since some features are not compatible from one backend to another. Sometimes, it is necessary to separate the compilation processes in order to produce static or dynamic libraries, which are then assembled to create a fat binary. A fat binary is a host executable that includes embedded device code for one or more target devices. This is especially the case for FPGA compilation.

5.2 Compiling for FPGA

FPGAs differ significantly from CPUs and GPUs in several ways. Regarding compilation, compared to CPUs and GPUs, generating an image for an FPGA is a very time-consuming process. Generally, this can take several hours. For this reason, the ahead-of-time (AOT) compilation flow is used for FPGAs.

In the AOT compilation flow, the device code is translated to SPIR-V and then processed by the device-specific backend in the host compiler, with the resulting device code being embedded in the generated fat binary. In this process, the target device must be specified at compilation time, which provides less flexibility than the just-in-time (JIT) compilation used for CPUs and GPUs. This trade-off between compilation time and deployment flexibility is a fundamental characteristic of FPGA development workflows, where the lengthy synthesis process makes runtime compilation impractical for most applications.

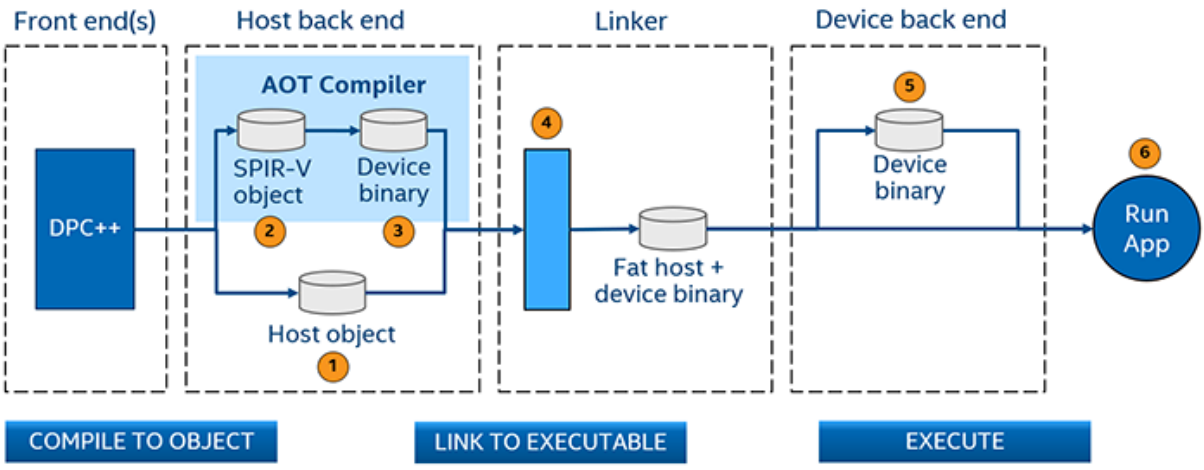


Figure 21: AOT compilation flow [11]

There are four types of compilation for FPGAs, which are summarized in Table 7.

Target	Output	Description
Emulator	x86-64 binary (seconds)	Compiles the FPGA device code to the CPU. Use the Intel® FPGA Emulation Platform for OpenCL™ software to verify the SYCL code's functional correctness.
Optimization Report	RTL + FPGA reports (minutes)	Compiles the FPGA device code to RTL and generates an optimization report that describes the structures generated on the FPGA, identifies performance bottlenecks, and estimates resource utilization. This report will include the interfaces defined in the selected Board Support Package.
Simulator	RTL + FPGA reports + x86-64 binary (minutes)	Compiles the FPGA device code to RTL and generates a simulation test bench. Use the Questa*-Intel® FPGA Edition simulator to verify the design.
FPGA Hardware	Quartus Place & Route + FPGA reports + x86-64 host binary (Multiple Hours)	Compiles the FPGA device code to RTL and compiles the generated RTL using Intel®

Table 7: Compilation targets and their characteristics [9]

The typical workflow for FPGA development involves going through all the different compilation types and optimizing or correcting the code at each stage as needed.

5.3 Compilation Flow

To compile both the CPU and FPGA, it was necessary to separate the code into two different files. One reason for this is the incompatibility between the compilation flags required for the CPU and those needed for the FPGA. Another reason is efficiency: if the FPGA kernel does not change (which is most often the case), it is not necessary to recompile it with the rest of the code.

In this compilation flow, the device kernels are built first, after which the host source code is compiled and linked with the device kernels (provided as a library). Figure 22 shows an example of separate compilation using the `vector-add` example, where two kernels, one CPU and one FPGA, are built separately before being linked together. This type of compilation was used

for compiling the OneJoin implementation, but not for the Tag Matcher, since it only uses a single queue.

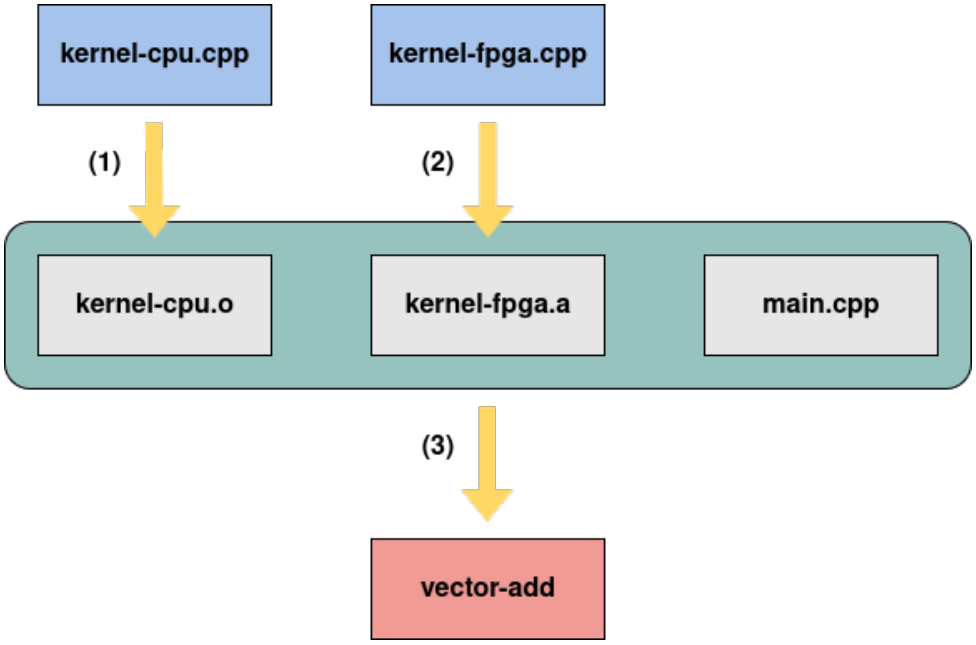


Figure 22: Schematic of the compilation flow [12]

```
#(1) Generate kernel-cpu.o (object file)
icpx -fsycl -fPIC -fsycl-targets=spir64 -c kernel-cpu.cpp -o kernel-cpu.o
#(2) Generate kernel-fpga.a (static lib)
icpx -fsycl -fintelfpga -fsycl-link=image -DFPGA_HARDWARE -Xshardware -
Xstarget=a7sk_usm kernel-fpga.cpp -o kernel-fpga.a
#(3) Generate executable from main.cpp and both lib and object files
icpx -fsycl -fintelfpga -o vector-add main.cpp kernel-cpu.o kernel-fpga-
hw.a
```

Code 9: Command lines of the compilation flow [12]

1. This step aims to produce a standard machine object compatible with the CPU, for the parts of the code that will run on the host processor. The `-fsycl` option enables SYCL C++ compilation. The `-fPIC` option tells the compiler to generate position-independent code for the host portion of the device code, so that pointers in the shared library reference global addresses. The `-fsycl-targets=spir64` option specifies that the target is a CPU device.

2. Here, the FPGA kernel is compiled and packaged into a static library intended to be linked and later programmed onto the board. The `-fintelfpga` option enables the Intel FPGA toolchain for bitstream generation. The `-fsycl-link=image` option tells the compiler to generate an FPGA bitstream, making it ready to be linked and used on an FPGA board. The `-DFPGA_HARDWARE` flag sets a compiler variable used to determine which `device_selector` to use, in this case `fpga_selector_v`. Options prefixed with `-Xs` are passed to the FPGA backend; for example, `-Xshardware` instructs the compiler to target FPGA hardware, and `-Xstarget=a7sk_usm` specifies the target FPGA device family.

3. At this stage, the linker combines all the pieces: the host code (`main.cpp`), the CPU objects, and the FPGA archive, into a single executable. Since the FPGA library is static, the `-fintelfpga` flag should be included in the compile command.

If needed, the `kernel-fpga.cpp` can be compiled as a GPU kernel for a GPU device. Using then the `-fsycl-targets=nvptx64-nvidia-cuda` flag combined with the right backend, the executable would be able to run on GPU.

Chapter 6 : Implementation Details

6.1 Tag Matcher

a) Dataflow and Parallelization

The Tag Matcher algorithm is divided into four stages: decoding references (intact tags) and computing Peq, decoding queries, computing Myers' algorithm, and collecting results. These stages are organized to achieve the most optimal dataflow possible and to minimize data transfers between the device and the host.

In general, dataflow refers to a pragma that can be requested from the compiler. For example, on AMD platforms this is achieved with `#pragma HLS dataflow`, placed at the beginning of a function that one wishes to optimize. In DPC++, however, this is not the case: memory accesses via buffers and accessors automatically handle this optimization. Nevertheless, this does not affect the logic of the code itself, which is why it is necessary to design the implementation with the goal of achieving the most efficient dataflow possible.

Figure 23 lists the stages together with their respective dataflow:

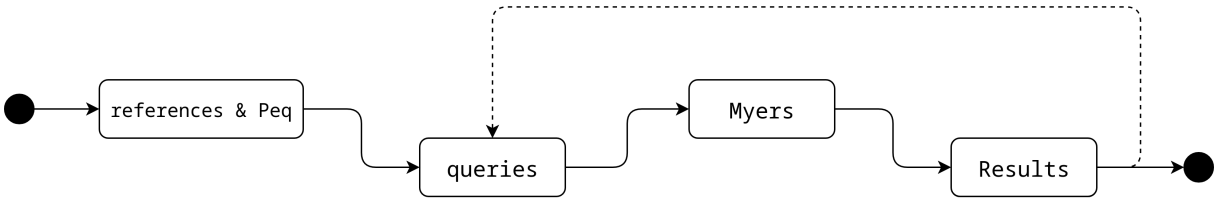


Figure 23: Tag Matcher algorithm dataflow

In this dataflow, the data arrives in a compressed form using the encoding described in Section 6.1.b and is stored sequentially in a one-dimensional array.

The first stage performs two tasks: decoding the references and computing their corresponding Peq matrices.

- Decoding is the process of extracting and isolating the references from the one-dimensional array.
- Precomputing the Peq matrices makes the subsequent stages more efficient. In Myers' algorithm, the first string is reused in every iteration of the main loop, but this is not the case for the second string, in our case, the references. Instead, the references are only required during the creation of the Peq table, which can therefore be precompiled before the main loop begins.

In our implementation, all references are sent once in order to compute their Peq matrices. These matrices are then reused in the following stages, such as during the comparison step. This approach reduces data transfers, optimizes device memory usage, and avoids recomputing Peq matrices multiple times, since only these precomputed matrices are kept in device memory.

The second stage, similarly to the first one with references, extracts the tags from the one-dimensional array to prepare them for the next step. Once a tag is extracted, it is stored in cache, as it will be heavily used during the Levenshtein distance computation stage. This caching accelerates subsequent accesses to the tag.

The third stage performs the computation of the Levenshtein distance using Myers' bit-parallel algorithm. This stage benefits from several optimizations, and its implementation is detailed later in Section 6.1.b.

The final stage processes the results obtained from the Levenshtein distance computation. It checks whether the computed distance falls within the required thresholds. This stage is discussed in more detail in Section 6.1.c.

It should be noted that all these stages are executed on the device. Specifically, the first stage is dispatched to the device using a `single_task`, while the remaining stages are dispatched using `parallel_for`.

b) The Myers' Bit-parallel Algorithm

During the semester project, a C++ implementation of Myers' algorithm was used. This implementation was developed by Fujimoto Seiji, author of *Polyleven*, a Python library designed to compute the Levenshtein distance. In his 2020 study, Fujimoto implemented both the Myers and Wagner–Fischer algorithms in order to compare them. We reused his open-source implementation and adapted it to make it compatible with DPC++. Please note that the website is currently down.

The DNA alphabet, and therefore the tags, is composed of the nucleotides A, C, G, and T. To transfer this alphabet efficiently to the target device, it was preferable to create a dedicated encoding instead of directly passing characters of type `char`. Since there are four possible values, the encoding can be represented on two bits, as follows:

Nucleotide	Encoding
A	00
T	01
G	10
C	11

Table 8: Binary encoding of nucleotides for device processing

This encoding is managed by the `DNAEncoding` class, which allows on-the-fly encoding and decoding of tags.

Tags are made up of 27 nucleotides. With this encoding, the bit length of a tag is 54 bits. A tag can therefore be stored and manipulated in a `uint64_t`, which is natively available in C++. However, when DNA strands undergo mutations, there are cases where a tag may extend to more than 32 nucleotides. In this case, the encoding leads to an overflow of the `uint64_t` variable.

To address this issue, the `DNABitVector128` class was implemented. This class relies on an array of two `uint64_t` values in order to store an entire tag in case of overflow. To make the Myers algorithm compatible with this class, all the operations required by the algorithm had to be reimplemented. Code 10 shows an example of the overwritten bitwise OR operator.

```
DNABitVector128 operator|(const DNABitVector128& other) const {
    DNABitVector128 result;

    #pragma unroll
    for (int i = 0; i < NUM_WORDS; i++) {
        result.words[i] = words[i] | other.words[i];
    }

    return result;
}
```

Code 10: Function bitwise OR overwritten with an unroll function

c) Levenshtein Distance Considerations

During the comparisons between input tags and reference tags, an early-stopping mechanism was implemented. When a comparison returns a Levenshtein distance lower than the threshold defined by the constant `MIN_LEVENSHTEIN_DISTANCE`, the tag is considered to have found its corresponding reference. At this point, the algorithm stops checking the remaining references for that tag and proceeds to the next one. This slightly speeds up the execution.

On the other hand, `MAX_LEVENSHTEIN_DISTANCE` is a constant that defines whether a Levenshtein distance is small enough to be considered valid. In other words, if a tag is too distant (greater than `MAX_LEVENSHTEIN_DISTANCE`) from all references, it will not be assigned any reference. As a consequence, the algorithm may drop tags that have undergone too many mutations.

A tag may sometimes be equidistant from several references. In such cases, all these references are taken into account, creating multiple tag–reference pairs. This ensures that the correct tag–

reference pair is generated. However, it also produces additional false pairs, which increases the number of wrong output pairs and thereby slightly decreases the precision of the algorithm. This is not problematic, since these false pairs are removed in the subsequent processing steps.

d) FPGA Optimization

During the implementation of Tag Matcher, several optimizations were introduced for an FPGA device. The two most important optimizations are the use of pragmas to fully exploit the high parallelism offered by FPGAs.

In addition to the previously discussed dataflow, two pragmas were added in this implementation: unroll and pipeline. The unroll pragma was applied to expand loops, such as those used in the DNAEncoding class. An example of the use of `#pragma unroll` is shown in Code 10.

The pipeline pragma is very useful for reducing the latency of certain loops, as well as for optimizing hardware utilization. By enabling pipelining on Non-task Kernels (parallel_for) using the -Xsauto-pipeline flag, several loops were successfully pipelined. Those are listed in Section 7.2.c.

6.2 OneJoin

a) oneoligo-a7sk

The OneJoin algorithm is available on GitHub under the repository name *oneoligo*, with two contributors: *Eug9* and *raja-appuswamy*, who are respectively the authors of the paper, *Eugenio Marinelli* and *Raja Appuswamy* [19]. This implementation served as the basis for our project. However, some modifications were necessary.

Since running on an FPGA requires ahead-of-time (AOT) compilation, certain parts of the code needed to be adapted. For example, the device selectors used for the queues or the Makefile had to be adjusted to properly target the chosen device during compilation. At first, this was handled with a simple patch that could be applied directly to the existing code.

```
#if defined(FPGA_SIMULATOR)
  using DeviceSelector = sycl::ext::intel::fpga_simulator_selector;
#elif defined(FPGA_HARDWARE)
  using DeviceSelector = sycl::ext::intel::fpga_selector;
#else // if defined(FPGA_EMULATOR)
  using DeviceSelector = sycl::ext::intel::fpga_emulator_selector;
#endif
```

Code 11: Code enabling selection of the appropriate queue

Another issue arose from the fact that the original implementation of *oneoligo* grouped all the accelerated kernels together in the same source file, making it difficult to compile simultaneously for both CPU and FPGA. To enable this, one of the stages of OneJoin, included in a kernel, was extracted from the main file and placed in an external file. This allowed it to be compiled with FPGA-specific flags instead of CPU compilation flags.

Along with other FPGA related adaptations, this represented a significant enough change to justify creating a dedicated fork of *oneoligo*, named *oneoligo-a7sk*. This fork enabled the development of a hybrid CPU–FPGA architecture, and is included as a sub-module within the GitLab repository of this project.

b) Parallelized Part

The part we decided to extract and parallelize was the kernel responsible for creating the buckets, namely the `create_buckets` function. The main reason this part was chosen is that, within the OneJoin pipeline, LSH bucketing constitutes one of the main bottlenecks in the filtering phase. Indeed, in OneJoin, the combination of the embedding and LSH steps accounts for more than 90% of the total execution time, with LSH bucketing being the most computationally expensive component of the filtering process [14].

The `create_buckets` function iterates in parallel over each triplet (batch, string index, replication index) and each LSH hash function to transform the sampled positions of the “embedded” sequence into a bucket identifier. For each element, it stores the tuple (replication index, hash

function index, bucket ID, string index, replication index). Each kernel instance performs exactly the same operations, without any dependencies between them. It is therefore an ideal function to measure and demonstrate the impact of parallelism.

Chapter 7 : Results and Performance

7.1 Hardware Used

The tests were carried out using three different devices: a CPU, a GPU, and an FPGA. The *CPU is an Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz* with an x86_64 architecture. It includes 8 threads distributed across 4 physical cores (2 threads per core). Its base frequency is 1.30 GHz, with a maximum frequency of up to 3.9 GHz.

The GPU is an *NVIDIA GeForce GTX 1660 Ti Mobile*. It comes with 6 GB of dedicated video memory. This GPU is designed for 3D graphics tasks as well as for parallel computations using CUDA. It is intended for use in a mobile environment.

The FPGA has already been presented in Chapter 3.

7.2 Tag Matcher

The tests were conducted on two main aspects: the accuracy of the algorithm and its speed. They were performed on a dataset of 897′249 reference tags. To simulate the tags obtained from DNA sequencing, 100′000 tags were generated from these references. These simulated tags were then subjected to 2% insertions, 4% deletions, and 4% substitutions.

The total theoretical number of comparisons to perform amounts to 879′249′000′000 (almost 9×10^{11}). In practice, however, this number is lower, thanks to optimizations added to the algorithm, such as early-stopping.

a) Precision

A good way to measure the accuracy of an algorithm is to make a confusion matrix. This matrix shows the number of True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN).

Within this context, it is rare to encounter FNs and impossible to encounter TNs. FNs may occur when none of the tag-to-reference comparisons result in a Levenshtein distance below `MAX_LEVENSHTEIN_DISTANCE`. However, the way tags are generated, combined with the chosen value of `MAX_LEVENSHTEIN_DISTANCE`, makes such cases impossible. Therefore, it is more relevant to focus primarily on TPs and FPs, which can be regarded as correct predictions and incorrect predictions, respectively. Those numbers are displayed in Figure 24.

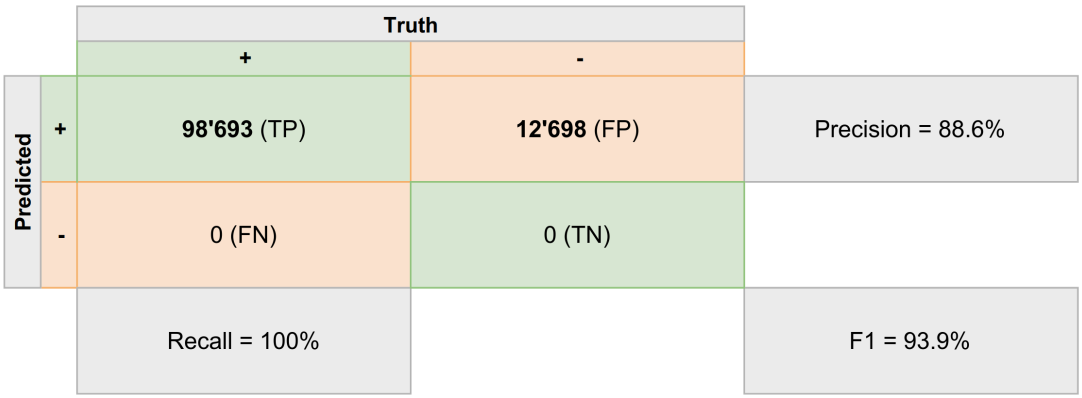


Figure 24: Confusion matrix of the Tag Matcher algorithm

To obtain a more balanced indicator of performance, it is common to use derived metrics. Precision is the proportion of correctly identified pairs among all the predicted pairs.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall is the model's ability to retrieve all the relevant pairs present in the expected results.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The F1-score is the harmonic mean of precision and recall, useful when seeking a trade-off between the two.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Here, a precision of 0.8860 means that 88.6% of the positive predictions are correct. A recall of 1.0000 (100%) indicates that the algorithm produced a prediction for every input, which can also

be observed from the fact that there are no false negatives (FN). An F1-score of 0.9396 reflects a good balance between the high precision (88.6%) and the perfect recall (100%), demonstrating very strong overall performance.

It is also worth noting that with 100′000 inputs, we end up with 111′391 outputs. This increase of 11′391 outputs (approximately 11%) is explained by the equidistant references discussed in Section 6.1.c. When a tag generates multiple outputs in this way, all but one of these outputs are false predictions, which then make up the majority of the false positives (FP).

Nevertheless, this approach still allows us to recover 98′733 of the 100′000 original tags. This corresponds to nearly 99% tag recovery, which is a remarkable result.

b) Speed

Regarding the speed of the algorithm, three devices were tested and compared in order to highlight performance differences and determine which one offers the best average time per comparison. Using the same dataset, the chart in Figure 25 illustrates the performance of the Tag Matcher algorithm.

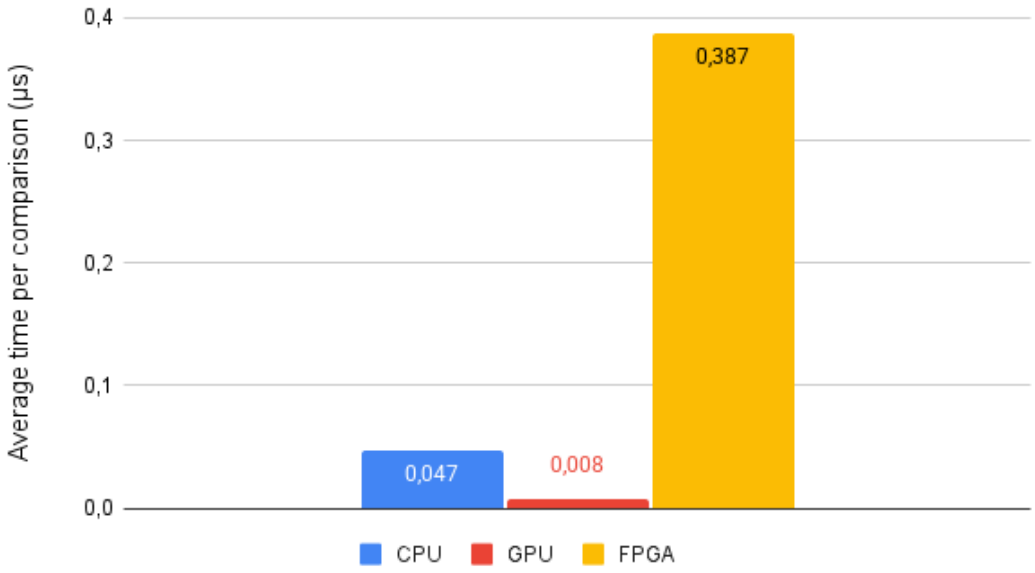


Figure 25: Average time per comparison of the Tag Matcher algorithm across three devices

The GPU significantly outperforms both the CPU and especially the FPGA for the task of tag comparison, achieving each comparison about 6 times faster than the CPU and nearly 50 times faster than the FPGA. In this context, the GPU demonstrates an extremely favorable performance-to-cost-to-time ratio. The CPU remains a versatile solution, but it falls far short of the GPU's efficiency for this kind of large-scale operation.

The FPGA result, however, is quite surprising. This type of hardware, when combined with such algorithms, would normally be expected to deliver solid performance. This outcome highlights a critical point: the FPGA implementation and hardware optimization are not yet properly refined.

c) FPGA Resource Usage

The report generated during the FPGA compilation of the Tag Matcher algorithm provides various insights into the FPGA's utilization. The key metrics we are interested in are :

1. the utilization of FPGA resources
2. the memory access latency
3. the optimizations that may have been applied

	Device	Static Partition	Kernel System
ALUTs	1′825′600	561′690 (31%)	59′211 (3%)
FFs	5′568′080	2′246′760 (40%)	137′172 (2%)
RAMs	13′272	3′500 (26%)	973 (7%)
MLABs	45′640	0 (0%)	932 (1%)

Table 9: Tag Matcher FPGA resource usage

Table 9 shows the resources utilized by the Tag Matcher algorithm on the FPGA. A rather surprising observation is that the available resources are never fully used. The design only occupies a small portion of the total resources, which suggests that the algorithm is not well optimized for FPGA and that there is still room to increase parallelism or further improve pipelining. It can also be observed that the kernels use a single compute unit. This lack of parallelism may partly explain the low resource utilization rate.

Unlike fixed architectures (CPU/GPU), FPGAs require specific manual optimizations that are not automatically performed by the compiler. Generally, manual interventions like pragmas or other attributes are needed to guide the compiler [20]. Furthermore, contextual analysis by the compiler is limited. It cannot always determine the best optimization strategies for a given algorithm, nor balance the trade-offs between latency, throughput, power consumption, and resource utilization [12]. These trade-offs also require manual tuning.

It should also be noted that the Intel oneAPI DPC++ compiler tends to adopt a conservative approach. It prioritizes stability, favoring a design that reliably works over one that might be faster but less reliable. In addition, the compiler may sometimes infer false loop-carried dependencies, leading to unnecessary serialization of certain operations.

Loop Description	Pipelined	fMAX (MHz)	Latency
Decoding Queries (line 499)	No	480.00	802
Initialisation of the best match indices (line 355)	No	480.00	13
Myers main loop (line 531)	No	480.00	871
Loop to decode references (line 434)	Yes	480.00	804
Initialisation of `Peq` (line 440)	Yes	480.00	19
Construction of `Peq` (line 462)	Yes	480.00	818

Table 10: Tag Matcher loops details

Table 10 illustrates the efficiency of some of the main loops in the Tag Matcher implementation. It originally also included all the loops for which the `#pragma unroll` directive was used. These loops were all correctly unrolled and optimized without latency. Since they are too numerous, they do not appear in this version of the table.

Except for the initialization loops, all the loops shown in Table 10 exhibit very high latency. This latency is primarily explained by their dependence on the FPGA's external global memory, namely, the DDR connected through the AXI controller. Each memory access ("Global Memory Load") requires a full round trip to the DDR, including burst phases and the serialization/

deserialization of data. This typically results in several cycles of delay. Moreover, given the implementation of the algorithm and the absence of explicit grouping directives, these loops generate scattered memory accesses rather than optimized bursts, further worsening the latency.

Regarding clock frequency, all elements operate at the same 480MHz, indicating that none of the loops were a frequency bottleneck. In fact, since the final kernel clock is set at 400MHz, all loops could theoretically go beyond this frequency, meaning that the actual bottleneck lies elsewhere.

7.3 OneJoin

Like Tag Matcher, OneJoin was evaluated for its accuracy and speed. These metrics were already tested in the paper *OneJoin: Cross-Architecture, Scalable Edit Similarity Join for DNA Data Storage Using oneAPI*, but it is important to validate them again on the project's data.

Ideally, this algorithm would have been tested on a dataset of 65′280 entries, since the project's pipeline, using Tag Matcher, initially divides the inputs into groups of 65′280. Unfortunately, as highlighted in Section 2.2, this was not possible. Instead, a dataset of 7′500 entries was used. Table 4 lists all the parameters of this dataset.

a) Accuracy

To analyze the accuracy of OneJoin, it was first necessary to evaluate the algorithm's output and adjust its parameters accordingly. Table 11 summarizes the most important parameter values that were finally selected.

Parameter	Value	Brief Description
NUM_STR	24	independent CGK embeddings per read
NUM_HASH	28	hash functions (OR-construction) per embedding
NUM_BITS	11	bits sampled inside each hash (AND-construction)
K_INPUT	48	edit-distance threshold passed to the verifier
SHIFT	16	edit-band stride used when slicing strings during embedding

Table 11: Recall of the most important OneJoin parameters

To compute the accuracy of OneJoin, several datasets with the parameters presented in Table 4 were generated. The total number of DNA strands, equal to 7′500, was obtained from 250 original DNA strands, each duplicated 30 times and subsequently modified with a 10% error rate. To achieve full accuracy, the algorithm should be able to re-identify the 250 groups corresponding to the original strands in each dataset.

In practice, this is exactly what was observed: OneJoin successfully recovered all 250 groups corresponding to the original strands in every dataset. Therefore, the algorithm achieved 100% accuracy.

b) Speed

On our dataset, the average runtime of this algorithm on CPU, GPU, and FPGA is approximately 3.5 seconds. However, this measure is not very relevant to the objectives of this project. In our case, comparing the execution speed of the parallelized part (specifically, the bucket generation) provides a more informative basis for evaluating device utilization.

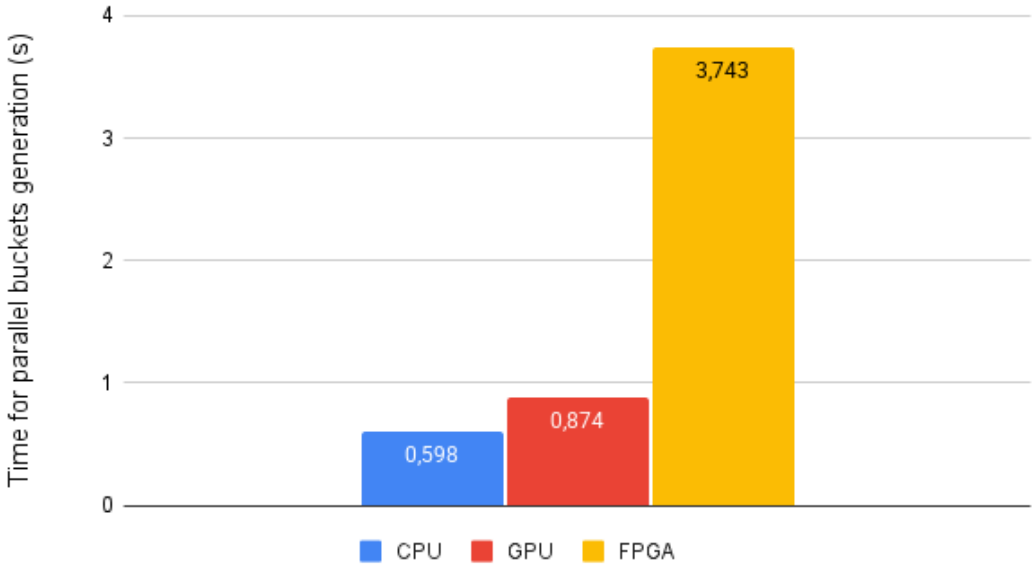


Figure 26: Execution time for parallel bucket generation across three devices

On our dataset, the CPU generates the buckets in 0.598 seconds, outperforming the GPU, which takes 0.874 seconds. The FPGA, however, is noticeably slower, with a runtime of 3.743 seconds.

The CPU proves to be the fastest for this specific task. This result may seem surprising, since one would usually expect a GPU or an FPGA to outperform the CPU on parallelized tasks. It suggests that, in our implementation and for this type of data, bucket generation leverages the CPU architecture more effectively than that of the GPU or FPGA. This difference may come from either the code optimization or the intrinsic architectural characteristics of the devices.

c) FPGA Resource Usage

Similar to the Tag Matcher, this section examines FPGA resource usage, latency, and applied optimizations. Both sections share many common aspects, including the explanation of the results, which are therefore not repeated here.

	Device	Static Partition	Kernel System
ALUTs	1′825′600	561′690 (31%)	68′982 (4%)
FFs	5′568′080	2′246′760 (40%)	106′675 (2%)
RAMs	13′272	3′500 (26%)	538 (4%)
MLABs	45′640	0 (0%)	81 (< 1%)

Table 12: OneJoin FPGA resource usage

Since the FPGA and the platform used for compiling both algorithms presented in this chapter are identical, it can be observed that the resources as well as for the static partition, are the same for both algorithms. The resource usage by the kernels here is also very minimal. The report further indicates that the number of compute units is one.

Loop Description	Pipelined	fMAX (MHz)	Latency
(line 96)	No	480.00	1618

Table 13: OneJoin loops details

The generation stage contains only one loop, in this case, it could not be optimized. As with the Tag Matcher algorithm, the latency observed is due to inefficient memory access management, specifically at the DDR level. The latency of 1618 cycles occurs because there are two memory accesses of approximately 800 clock cycles each, with one depending on the other.

The OneJoin code was designed with two types of devices in mind: CPUs and GPUs. It was not originally intended to be used with FPGAs, which require particular attention to memory accesses as well as optimization strategies. It has been shown that SYCL code optimized for one platform often performs poorly on different architectures. As noted by Franquinet (2023) in his analysis of SYCL performance portability : “As a result, the performance of the FPGA is not as good as on the CPU and GPU.” [21] This highlights the low level of performance portability and underlines the need for architecture-specific optimizations.

The performance portability challenges between GPUs and FPGAs stem from their different optimization strategies. First, while both architectures support multiple forms of parallelism, they emphasize different approaches. FPGAs excel at deep pipelining and spatial parallelism through custom hardware layouts, whereas GPUs prioritize massively parallel SIMT execution with thousands of concurrent threads. Second, memory bandwidth and access patterns differ significantly. Current FPGA boards typically offer lower off-chip bandwidth but provide flexible, custom memory hierarchies, while GPUs offer high bandwidth optimized for regular access patterns. Third, synchronization mechanisms have different performance characteristics. While GPUs handle synchronization primitives efficiently through dedicated hardware, FPGAs may require custom synchronization logic that can introduce stalls, though modern implementations provide optimized solutions [22].

7.4 Proposed Improvements

The low resource utilization and high latencies observed in the main loops indicate that the current designs do not efficiently exploit the capabilities of FPGA hardware. This section proposes targeted optimization strategies based on Intel oneAPI best practices. These could not be implemented within the scope of this project due to limited availability of resources.

a) Memory Access Optimisations

The observed latencies (802–1618 cycles) are primarily caused by inefficient accesses to the external DDR memory. As suggested in the *FPGA Optimization Guide for Intel oneAPI Toolkits*, restructuring memory access patterns to favor contiguous rather than scattered accesses would reduce the design's latency. Another approach would be to leverage local memory. Moving critical data from DDR to on-chip memory using local buffers avoids the performance penalty of slow memory transactions. This method can reduce access latency by several orders of magnitude.

A model proposed in the paper *Improving the Efficiency of OpenCL Kernels through Pipes* [22] suggests splitting the computation into two separate kernels: one dedicated to memory access and the other to computations. To communicate, these two kernels would use pipes. In the paper, this model leads to performance improvements, in some cases significant. This could be a solution to the memory access problems.

b) Improving the Initiation Interval (II)

Non-pipelined loops represent the main bottleneck. To reduce the II, one effective technique is to eliminate false dependencies. The attribute `[[intel::ivdep]]` can be used to instruct the compiler to ignore unnecessary dependencies between loop iterations, thereby enforcing pipelining.

c) Resource Utilization

Given the currently low utilization of FPGA resources, an obvious strategy would be to replicate the Myers compute unit to process multiple sequences in parallel, thus exploiting a greater share of the FPGA's available resources. In this way, several instances of the main kernel could be executed concurrently, each handling a different portion of the input data.

Chapter 8 : Conclusions and Perspectives

This project explores FPGA-based hardware acceleration for the implementation of Levenshtein distance, applied to the clustering of DNA strands within the DNAMIC project. We emphasize the use of tools such as SYCL, which enables efficient execution across a wide range of hardware platforms, including CPUs, GPUs, and FPGAs, without requiring platform-specific code modifications. Two approaches were studied and implemented: OneJoin and Tag Matcher.

OneJoin relies on a preprocessing stage using CGK embedding and Locality-Sensitive Hashing (LSH) to reduce the number of comparisons. In addition to accelerating the clustering of DNA strands, this algorithm achieves full cluster coverage on medium-sized datasets.

Despite OneJoin's speed, the Tag Matcher was designed to perform a pre-clustering step based on fixed-length tags found within DNA strands. This pre-clustering stage groups sequences before applying OneJoin to them. By feeding OneJoin with smaller subsets of data, its execution speed is significantly improved. Despite the insertions, deletions, and substitutions occurring in DNA sequences, this module achieves a recall of 99% with a precision of 88.6%.

Experimental results showed that the GPU is probably the best device for running the Tag Matcher algorithm, closely followed by the CPU. In the case of bucket generation in OneJoin, the CPU turns out to be the fastest this time, followed by the GPU. As for the FPGA, it performs much more slowly on both algorithms, partly due to the significant memory access latency and the relatively limited utilization of FPGA resources.

SYCL provides an important abstraction that simplifies development across heterogeneous architectures: the same C++ codebase can target CPUs, GPUs, or FPGAs interchangeably. This "marketing promise" of code portability drastically reduces development complexity and time, as there is no need to maintain multiple codebases or master different languages (such as VHDL/Verilog, CUDA, or OpenCL). However, in practice, code portability does not guarantee perfor-

mance portability. Without explicit optimizations and directives (such as pragmas, pipelining, dataflow, and others), FPGA kernels compiled from a generic SYCL codebase perform far below expectations. In other words, performance is portable, not the code: while it is easy to port a software codebase onto FPGA, fully exploiting its computational potential requires profiling and architecture-specific tuning. This was precisely the case encountered in this project.

Several areas for improvement were identified to unlock the full potential of FPGA acceleration. Most critically, memory access should be optimized by grouping transfers to DDR, thereby significantly reducing off-chip access latency. Another key strategy is reducing the initiation interval (II) of critical loops by adding compilation directives, enabling more effective pipelining than what is achieved through compiler pragmas alone. Furthermore, replicating the compute units of the core algorithms would increase throughput by allowing multiple kernel instances to run concurrently.

In conclusion, SYCL enables a unified implementation of Levenshtein distance across CPU, GPU, and FPGA, achieving a recall of 99% with OneJoin and Tag Matcher. While the GPU remains better for massive computing, the FPGA architecture offers significant potential if it is well optimized (memory access, pipelining, core duplication) to reduce latencies and increase the use of hardware resources.

Appendix 1 : Relevant links to the project

- git fpga-dna-strand-clustering, adrian.spycher : https://gitedu.hesge.ch/lg_bachelors/ts/2024/fpga-dna-strand-clustering
- git oneoligo-a7sk, adrian.spycher : https://gitedu.hesge.ch/adrian.spycher/oneoligo-a7sk

References

[1] PEARL-DNA, “State of the Art DNA Data Storage | DigNA Webinar Series (Webinar 1).” Accessed: Aug. 20, 2025. [Online]. Available: https://www.youtube.com/watch?app=desktop&v=gNv72a1ugiA

[2] “MinION portable nanopore sequencing device.” Accessed: Aug. 20, 2025. [Online]. Available: https://nanoporetech.com/products/sequence/minion

[3] P. Agarwal, “Program to Calculate the Levenshtein Distance in List in Python.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.scaler.com/topics/levenshtein-distance-python/

[4] “Wagner–Fischer algorithm.” Accessed: Aug. 20, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Wagner%E2%80%93Fischer_algorithm&oldid=1301922518

[5] G. Myers, “A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming.”

[6] T. Technologies, “Terasic - All FPGA Boards - Agilex 7 - Agilex 7 FPGA Starter Kit.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=142&No=1295&PartNo=2

[7] “OFS.” Accessed: Aug. 20, 2025. [Online]. Available: https://ofs.github.io/ofs-2024.3-1/

[8] “SYCL - The Dawn of a Unified Programming Model for Heterogeneous Modern C++ at SC19.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.khronos.org/blog/sycl-the-dawn-of-a-unified-programming-model-for-heterogeneous-modern-c-at-sc19

[9] “Home - SYCL.tech.” Accessed: Aug. 20, 2025. [Online]. Available: https://sycl.tech/

[10] “Introduction • Vitis High-Level Synthesis User Guide (UG1399) • Reader • AMD Technical Information Portal.” Accessed: Aug. 20, 2025. [Online]. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls

[11] “Intel® oneAPI Programming Guide.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/docs/oneapi/programming-guide/2023-2/overview.html

[12] W. Liang, N. Fujita, R. Kobayashi, and T. Boku, “Using Intel oneAPI for Multi-hybrid Acceleration Programming with GPU and FPGA Coupling,” in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region Workshops*, Nagoya Japan: ACM, Jan. 2024, pp. 69–76. doi: 10.1145/3636480.3637220.

[13] “Edit distance.” Accessed: Aug. 20, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Edit_distance&oldid=1299105700

[14] E. Marinelli and R. Appuswamy, “OneJoin: Cross-Architecture, Scalable Edit Similarity Join for DNA Data Storage Using oneAPI.”

[15] D. Chakraborty, E. Goldenberg, and M. Koucký, “Streaming algorithms for embedding and computing edit distance in the low distance regime,” in *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, Cambridge MA USA: ACM, Jun. 2016, pp. 712–725. doi: 10.1145/2897518.2897577.

[16] “Locality Sensitive Hashing (LSH): The Illustrated Guide | Pinecone.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.pinecone.io/learn/series/faiss/locality-sensitive-hashing/

[17] “Agilex™ 7 FPGA and SoC FPGA Overview.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.altera.com/products/fpga/agilex/7

[18] “Intel oneAPI DPC++/C++ Compiler Handbook for FPGAs Overview.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/2025-0/fpga-handbook.html

[19] E. Marinelli, “Eug9/oneoligo.” Accessed: Aug. 20, 2025. [Online]. Available: https://github.com/Eug9/oneoligo

[20] “FPGA Optimization Guide for Intel® oneAPI Toolkits.” Accessed: Aug. 20, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/optimization-guide/2023-1/overview.html

[21] J. Franquinet, “Performance portability analysis of SYCL with a classical CG on CPU, GPU, and FPGA.”

[22] M. E. Zarch and M. Becchi, “Improving the Efficiency of OpenCL Kernels through Pipes.”